

Avvenuti

Localizzazione

Parla della localizzazione in generale

Sapere dove si trova un dispositivo in un particolare istante temporale e associare tali informazioni (istante temporale e collocazione) a particolari eventi.

Un nodo può essere in tre possibili stati differenti:

1. **Unknown:** Non conosce la sua posizione.
2. **Settled:** Ha determinato la sua posizione.
3. **Beacon:** Conosce la sua posizione (GPS) ed invia periodicamente beacon contenente la sua posizione.

La localizzazione si basa su un algoritmo di localizzazione che consente agli unknown node di **stimare** la posizione e quindi diventare settled.

Le quattro architetture per la localizzazione dei nodi

Remote-positioning

La posizione di ogni nodo unknown è calcolata da un dispositivo centrale (hub).

1. Il target trasmette un segnale.
2. Vengono ricevuti dalle ancore e inoltrati ad un hub.
3. Lo hub calcola la posizione del target.

Le computazioni consumano energia solo nello hub, mentre dai target e dalle ancore è spesa energia per la comunicazione.

La posizione è nota solo alla base station e non ai target.

Self-positioning

La posizione è calcolata dal nodo stesso, sfruttando i segnali periodici che riceve dalle ancore.

1. Periodicamente ogni ancora invia un beacon.
2. I target ricevono il segnale e stimano la posizione.

La posizione dei target è nota solo a loro e a nessun'altro nodo.

GPS → Tecnica di Self-positioning: ogni dispositivo calcola la propria posizione usando il satellite come ancora.

Se vogliamo che entrambi i nodi target e la base station conoscano le posizioni dobbiamo utilizzare la terza/quarta topologia.

Indirect Remote-Positioning

Calcolata dal target, poi inviata allo hub.

Indirect Self-Positioning

Calcolata dallo hub, poi inviata al target.

Differenza tra approccio centralized e localized

La differenza sta nel **dove vengono fatti i calcoli**.

Centralized

Lo hub stima la posizione del target utilizzando i segnali inoltrati dalle ancore.

- Più preciso.
- Meno scalabile.
- Se il numero di ancore non è sufficiente a coprire l'area è necessario introdurre il multi-hop.

Localized

Il target stima la sua posizione sfruttando i beacon inviati dalle ancore.

- Più semplice.
- Più energy-hungry.

Algoritmi range-based

1. **Ranging phase:** target calcola il range dalle ancore vicine (da cui ha ricevuto un beacon).

Range → la stima della **distanza fisica tra un'ancora ed un target**.

2. **Estimation phase:** target utilizza le informazioni individuate per stimare la posizione (self-positioning).

Ranging phase

Il target riceve un beacon da almeno tre ancore. In ogni beacon è presente la posizione dell'ancora mittente.

Per ogni ancora stima (tramite uno dei metodi sotto) il range tra sé stesso e l'ancora.

RSSI – Self Positioning

L'interfaccia hardware del target riceve un beacon e **stima la potenza del segnale ricevuto**.

$$P_r(d) = \frac{P_t G_t G_r \lambda^2}{(4\pi)^2 d^2}$$

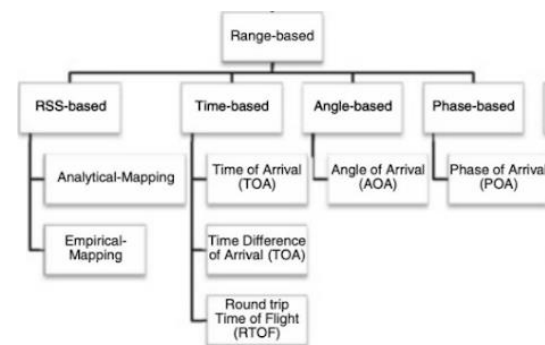
- P_t → Potenza del segnale trasmesso, in generale noto a tutta la rete (è lo stesso per ogni anchor).
- P_r → Potenza del segnale ricevuto, misurato tramite componenti hardware.
- G_t e G_r sono costanti di guadagno, rispettivamente del trasmettitore e del ricevitore (dipendono dalle antenne del sistema, quindi sono note a noi progettisti);
- λ → Un'altra costante.
- d → La distanza tra l'ancora e il nodo ricevitore, il valore che dobbiamo trovare.

La versione empirica (adattata al luogo di riferimento in cui lavorerà il sistema)

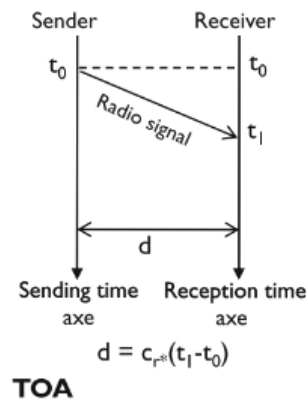
$$P_{RSSI} = \frac{X}{r^n}$$

Dove X è ricavato tramite esperimenti.

ToA – Indirect Self Positioning



Si basa sulla differenza tra tempo di invio (incluso nel messaggio) e tempo di arrivo di un beacon



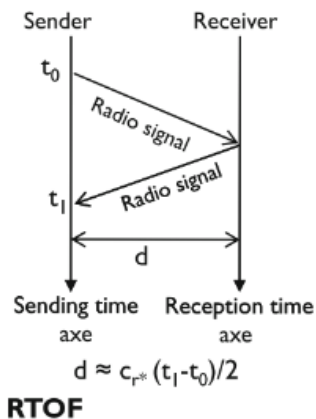
1. Le ancore inseriscono nel beacon l'istante $Time_{Send}$ in cui è stato trasmesso.
2. Il target calcola $ToA = Time_{Arrival} - Time_{Send}$
3. Il target calcola $d = c * ToA$

Problema -> Il clock del trasmettitore e del ricevitore devono essere precisamente sincronizzati.

Il GPS funziona bene perché i satelliti hanno un clock atomico (sincronizzati precisamente) quindi può essere calcolato con un errore piccolo.

RTOF – Indirect Remote Positioning

Se non è possibile avere clock sincronizzati.

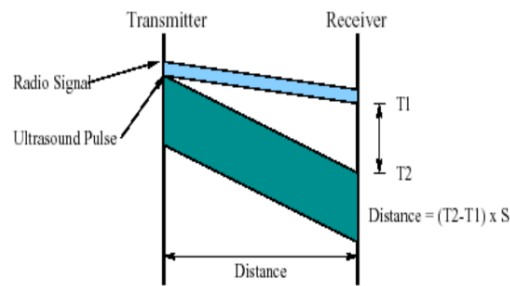


1. L'ancora invia il beacon a tempo t_0 .
2. Il target riceve e invia ACK, l'ancora lo riceve a tempo t_1 .
3. L'ancora calcola $d = \frac{c}{2} * (t_1 - t_0)$ e la invia al target.

TDoA – Indirect Self Positioning

Se non è possibile avere sincronizzati.

Si sfrutta la differenza temporale tra la ricezione del segnale trasmesso e un segnale composto da ultrasuoni. Usando il suono, l'errore diventa molto più piccolo rispetto a ToA, perché s è molto più piccolo di c , quindi non permette agli errori di esplodere.



1. L'ancora invia un **segnale radio** e un **segnale sonoro** allo stesso istante (subito dopo).
2. Il target riceve a (t_1, t_2) e calcola $d = s * (t_2 - t_1)$

AoA – Indirect Self Positioning

Le ancore trasmettono beacon e i target misurano l'angolo di arrivo.

Se riceviamo un segnale, utilizzando un riferimento (come una bussola che punta al nord) e conosciamo l'angolo di arrivo del segnale (relativamente al riferimento), possiamo disegnare un triangolo per stimare la posizione del mittente.

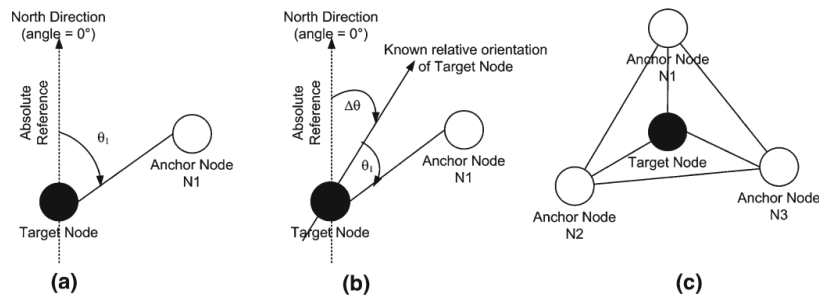


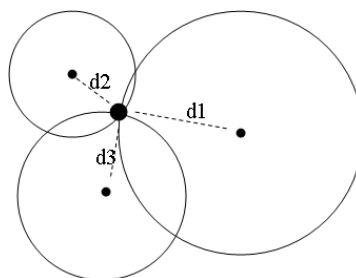
Fig. 7 AOA orientation concept. **a** Absolute orientation. **b** Relative orientation. **c** AOA with unknown orientation

Estimation phase

Il target utilizzerà la tabella costruita durante la ranging phase per stimare le proprie coordinate.

Trilaterazione

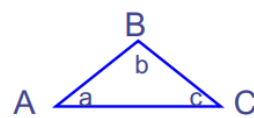
Si misura la distanza tra il nodo e almeno tre ancore, poi si tracciano tre circonferenze con raggio le tre distanze.



L'intersezione definisce l'area in cui si trova il target. La posizione è il **centro di gravità** di tale area

Triangolazione

Si misurano gli angoli tra le tre ancore grazie alle coordinate delle ancore. Utilizziamo questi angoli per delimitare il poligono e quindi l'area di ricerca.



Sines Rule

$$\frac{A}{\sin a} = \frac{B}{\sin b} = \frac{C}{\sin c}$$

Cosines Rule

$$C^2 = A^2 + B^2 - 2AB \cos c /$$

$$B^2 = A^2 + C^2 - 2AC \cos b /$$

$$A^2 = B^2 + C^2 - 2BC \cos a /$$

Multilaterazione

Generalizzazione dei metodi precedenti: si sfruttano tutti i beacon node presenti, non solo tre. Utilizzato in sistemi caratterizzati da elevata densità di beacons: maggiore è il numero di beacons, maggiore è l'accuratezza della stima.

Qual è lo svantaggio della RSSI?

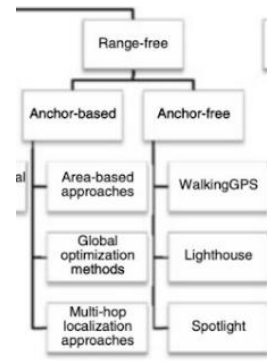
Sebbene sia economica e facile da implementare. Sia la versione nello spazio libero che quella empirica necessitano di una analisi dell'ambiente e, se l'ambiente cambia, possono risultare meno accurate.

Algoritmi range-free in generale

Problemi dei range-based

Precisi solo con una buona formula della distanza, ma spesso sono valide solo con assunzioni rigorose:

- **Radio range circolare:** nella realtà non è perfettamente circolare a causa dell'ambiente. Il target potrebbe essere al di fuori del range effettivo di un'ancora.
- **Necessaria una comunicazione simmetrica:** Se riceve il beacon da un ancora non è scontato che lo ACK arrivi.
- **Non devono esserci ostacoli tra i nodi e multi-path limitato:** introduce rumore nei segnali radio.
- **Hardware aggiuntivo:** come nel caso TDoA e AoA.



Range-free

Si individua la posizione senza calcolare la distanza precisa tra target e ancore ma solo con metriche di prossimità e connettività:

- **Anchor-based methods:** Sfrutta le ancore che conoscono la propria posizione (GPS o configurazione) e inviano dei beacon.
- **Anchor-free methods.**

Area-based Approach

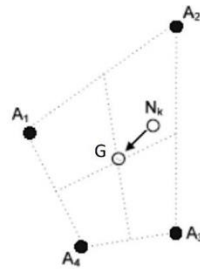
Approccio facente parte dei metodi anchor-based.

Dopo che un target ha ricevuto un certo numero di beacon, può stimare la propria posizione senza dover calcolare la distanza precisa rispetto alle ancore. Un esempio è il metodo del **centroide** oppure **APIT**.

Centroide

Dai beacon, il target ottiene le coordinate delle ancore con le quali **costruisce un poligono** con vertici $(A_1, A_2, \dots, A_n)$ le ancore. La posizione è il centroide N_k del poligono.

$$(x_g, y_g) = \left(\frac{\sum_{i=1}^n x_i}{n}, \frac{\sum_{i=1}^n y_i}{n} \right)$$



Vantaggi:

- Numero di ancore variabile.
- **Semplice** da implementare e a **basso consumo**.
- Poligono grande → precisione bassa.

Svantaggi

- **Poco accurato**: assume che il nodo si trovi all'interno dei range delle ancore.

APIT - Approximate Point-In-Triangulation

Approccio **area-based** dove l'area in cui si calcola il centro di gravità di un triangolo. Molto più preciso del centroide e l'errore rimane contenuto all'aumentare della densità dei nodi.

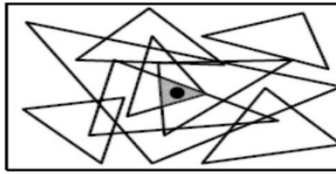
```
Receive location beacons  $(X_i, Y_i)$  from  $N$  anchors;  
 $InsideSet = \emptyset$ ;  
for each triangle  $T_i \in \binom{N}{3}$  triangles do  
  if Point-In-Triangle-Test  $(T_i) == \text{TRUE}$  then  
     $InsideSet = InsideSet \cup T_i$ ;  
  end if  
end for  
Estimated Position = CenterOfGravity( $\bigcap T_i \in InsideSet$ );
```

1. Il target riceve i beacon da tutte le ancore a portata (contiene le coordinate dell'ancora).
2. Si formano tutti i possibili triangoli combinando gruppi di tre ancore.

Coefficiente binomiale tra N e 3, con N il numero di ancore: $N_{Triangoli} = \frac{N!}{3!(N-3)!}$

3. **Per ogni triangolo** → APIT Test per verificare se il target si trova all'interno del triangolo.
4. Si conservano solo i triangoli in cui il nodo è interno e si calcola **l'intersezione**.
5. **Posizione** → **Centroide dell'intersezione**.

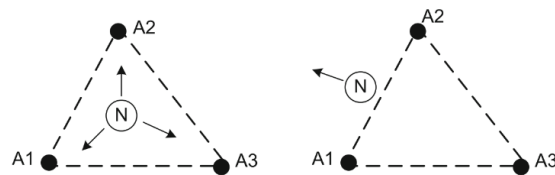
Meglio se le ancore sono distribuite in modo omogeneo, perché i triangoli saranno di dimensione omogenea.



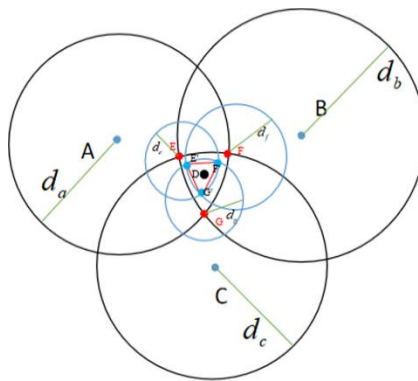
APIT Test – Perfetto

Nel test APIT, non si misurano vere e proprie distanze (come nei range-based). Si valuta la posizione relativa di un nodo rispetto a un triangolo formato da tre ancore.

Questa informazione può essere ottenuta un **RSSI grezzo** solo per capire se "si avvicina" o "si allontana" da un lato del triangolo, ma **senza** stimare una distanza precisa.



Supponiamo di avere le tre ancore e il target N_k , disegniamo oltre al triangolo, tre cerchi centrati nelle ancore e che passano per il target. Supponiamo anche che il target si muova.



- Se il target si sposta verso A → entra all'interno del cerchio di A1 ed esce dai cerchi di A2 ed A3.
- Se il target si sposta verso A e C → Esce dal cerchio di A2.

Se questo accade, il nodo capisce che è all'interno, altrimenti capisce che è all'esterno.

Test perfetto → Muovere il target lungo tutte le possibili direzioni.

- **Interno** → muovendosi in qualunque direzione, si avvicinerà sempre ad almeno un'ancora.
- **Esterno** → esiste almeno una direzione che lo porterà ad allontanarsi da tutti e tre le ancore.

Non è possibile muoversi in tutte le direzioni e non sempre i nodi si muovono secondo il nostro volere.

APIT Test Approssimato

1. Il target chiede ai suoi vicini (non ancore) di trasmettere la loro distanza (RSSI grezzo) rispetto alle tre ancore.

Ogni vicino n_1 , n_2 , n_3 , n_4 trasmette una tupla di tre distanze a N.

$D_{ni} = \{ \text{RSSI}_{ni_1}, \text{RSSI}_{ni_2}, \text{RSSI}_{ni_3} \}$

2. N calcola la sua distanza rispetto alle tre ancore.

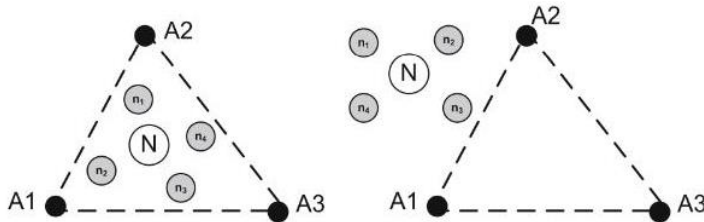
$D_N = \{ \text{RSSI}_{N1}, \text{RSSI}_{N2}, \text{RSSI}_{N3} \}$

3. Ricevute le distanze dai vicini fa un confronto con le proprie.

4. N è **interno** se **NON** esistono vicini con tutte e tre le distanze:

- Simultaneamente maggiori delle distanze di N.
- Simultaneamente minori delle distanze di N.

Confronto tra vettori, elemento per elemento.



- **Interno** → nessun vicino è più vicino/lontano da tutte e tre le ancore rispetto a N.
- **Esterno** → esiste un vicino più vicino/lontano a tutte le ancore rispetto a N.

Calcolo intersezione

Per rappresentare l'intersezione dei triangoli utilizziamo **grid scan**.

L'area è divisa in celle, ciascuna rappresentata da un valore numerico (inizialmente zero);

Le aree dei triangoli sono associate alle celle → ogni volta che PIT segnala che il target è all'interno di un triangolo incrementiamo le celle "toccate" da quel triangolo, se invece non è presente decrementiamo.

0	0	0	0	0	0	1	0	0	0
0	0	1	0	1	1	1	0	0	0
0	0	1	1	1	1	1	0	0	0
0	1	2	2	1	1	0	-1	0	0
1	1	2	2	1	1	0	-1	-1	0
0	0	2	2	2	1	0	-1	-1	-1
0	0	1	1	1	0	0	-1	-1	-1

Le celle con il valore massimo rappresentano l'area di intersezione.

Perché con il PIT approssimato si possono avere errori di stima?

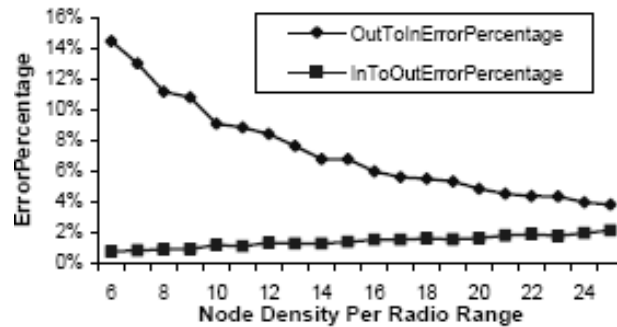
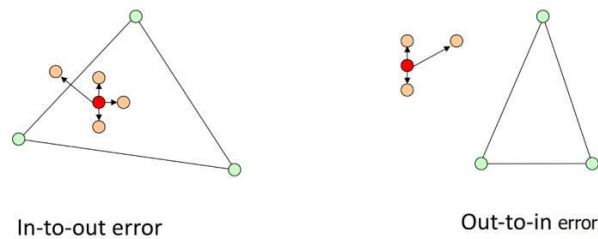
Ha errori perché è approssimato → Non è possibile muovere il target lungo tutte le possibili direzioni.

In-to-out error (aumenta con la densità dei nodi)

Target interno ma viene segnalato esterno (un vicino esterno). Le distanze del vicino risultino tutte maggiori di quelle del target, quindi verrà classificato esterno.

Out-to-in error (diminuisce con la densità)

Target esterno ma viene segnalato interno. Nessuno dei tre vicini è più lontano/vicino a tutte le ancore rispetto al nodo target.



DV-HOP in generale

Per APIT e centroide, assumiamo che il target sia all'interno del range di molte ancore.

Ma **una comunicazione one-hop non sempre è possibile**; occorre estendere la localizzazione oltre il range → **meccanismi di localizzazione multihop**.

Supponiamo di avere quattro ancore (A_1 , A_2 , A_3 , A_4) ed un target N .

Fase 1: Propagazione (tutti)

1. A_i invia un beacon B_i con le coordinate (X_i, Y_i) ed un **hop counter** h_i inizializzato ad 1.

$B_i = \{ (X_i, Y_i), h_i, c_i \}$

2. Un qualsiasi nodo che riceve B_i :
 1. Inserisce in una tabella i dati.
 2. h_i++
3. Inoltra il beacon ai propri vicini.

Ogni nodo riceve i beacon da tutte le ancore (inoltrati dai nodi intermedi) e nella tabella ha le coordinate e il numero di hop necessari per raggiungere qualsiasi ancora.

Fase 2 : Calcolo (ancore)

4. Ad ogni ricezione di un beacon un'ancora calcola il proprio correction factor e poi lo diffonde nei beacon.

$$c_i = \frac{\sum_j \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}}{\sum_j h_j}$$

c_i è la **average 1-hop distance**, ovvero la distanza fisica media "legata a un singolo hop"

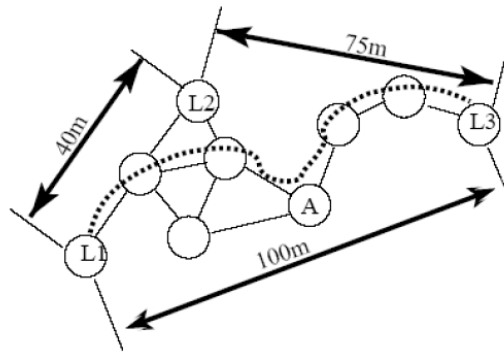
- **Numeratore** → Somma delle distanze euclidee tra A_i e tutte le ancore.
- **Denominatore** → Numero totale di hop.

Fase 3 : Calcolo posizione (target)

5. Ogni target riceve da ogni ancora il correction factor.

- Sceglie quello dell'ancora più vicina in termini di hop e lo usa per calcolare le distanze dalle ancore.

$$d_j = h_j * c$$



- Il target A riceve c_i di ogni ancora (nei beacon), sceglierà quello proveniente dalla più vicina (L2).

Più è vicino all'ancora, più è simile l'ambiente di trasmissione e quindi più affidabile sarà la stima.

- Stima la distanza da ogni ancora: moltiplica c_2 per il numero di hop necessari.

Il calcolo di questa distanza si rifà al teorema di pitagora.

L_1 computes the correction $\frac{100+40}{6+2} = 17.5$.

L_2 computes a correction of $\frac{40+75}{2+5} = 16.42$

L_3 a correction of $\frac{75+100}{6+5} = 15.90$.

A gets its correction from L_2

Distance to L1 = 3 x 16.42, to L2 = 2 x 16.42, to L3 = 3 x 16.42

- Stimata la distanza di almeno tre ancore è possibile usare la **triangolazione**.

Svantaggi di DV-HOPS:

- Il numero di ancore incide sulla stima;
- overhead nella comunicazione**, le ancore propagano molti messaggi;
- Assunzione che la distanza tra nodi distanti 1-hop siano uniformi** (isotropic and anisotropic networks, nell'ultima tipologia di rete la distanza fisica cambia in base alla direzione intrapresa).
- Le informazioni che utilizziamo per calcolare le posizioni sono basate anche sulla connettività.**

Come funziona la **trilaterazione**?

Almeno tre ancore, per poi costruire una tabella con quelle coordinate.

Dopo aver fatto ciò, si misura la distanza da essi (RSSI o ToA) ed essa verrà utilizzata come raggio di un cerchio centrato nell'anchor (questo per ogni anchor). L'intersezione tra questi cerchi o l'area di intersezione, se la stima non è precisa, sarà la posizione stimata del target.

Come funziona la **triangolazione**?

Almeno due ancore con antenne direzionali.

- Il target trasmette un beacon, che viene ricevuto dalle ancore.
- Le ancore misurano l'angolo di arrivo con AoA (rispetto a un riferimento come il Nord).
- Conoscendo la propria posizione e l'angolo ogni ancora traccia una direzione.
- Il punto in cui queste direzioni si intersecano è la posizione stimata del nodo.

DHT

Paradigma Publish-Subscribe in generale

Schema di comunicazione in cui i publisher non inviano messaggi a specifici subscriber ma anzi, il messaggio viene categorizzato con un topic e inviato ad un punto di rendez-vous.

Il punto di rendez-vous è il punto in cui un publisher pubblica e un subscriber può ricevere i messaggi.

Comunicazione Indiretta → Al subscriber non interessa sapere chi sono i publisher e viceversa. Il publisher **non specifica l'indirizzo** del destinatario: si limita a classificare il messaggio.

La rete fisica su cui viaggiano i messaggi prende il nome di **rete di distribuzione**.

Filtering

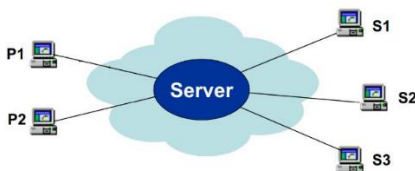
I subscriber riceveranno solo un sottoinsieme dei messaggi inviati, due categorie:

- **Topic-Based Filtering**
 - Messaggi associati ad un topic, detto anche “canale logico”, identificato da una chiave.
 - I subscriber riceveranno tutti i messaggi pubblicati sui topic che hanno sottoscritto.
 - **Complessità sul publisher** → deve definire i topic e classificare i messaggi.
- **Content-Based Filtering**
 - Oltre al topic, vengono considerate delle keywords nel messaggio.
 - Il subscriber riceve tutto, quest'ultimo accetta solo se gli attributi corrispondono a quelli definiti.
 - **Complessità sul subscriber** → deve definire e applicare i criteri di match per classificare i messaggi.

Descrivi le implementazioni del P-S e i loro pro e contro.

Client-Server

Semplice e centralizzata.



Il server è il rendez-vous: riceve advertisement dai publisher, memorizza i messaggi e in maniera asincrona li trasmette ai subscriber.

Complessità sul server: è responsabile della distribuzione dei messaggi:

- **Non è scalabile** con il numero di nodi.
- **Single point of failure.**

Ci sono metodi per risolvere i due contro, come la replicazione ecc.

Peer-to-Peer

Non esiste un controllo centralizzato e tutti i nodi sono in grado di svolgere tutte le funzioni, senza un'assegnazione di ruoli a priori.

- **Più scalabile** rispetto ad una Client-Server
- **Più complessa** soprattutto in presenza di un numero elevato di nodi con hardware diversi, potenza differenti.
- **Nodi inaffidabili:** Elevata dinamicità a livello hardware.

Lookup Problem → chiedere al sistema la posizione delle informazioni associate ad una certa chiave.

Il nodo sa che l'informazione è messa da qualche parte (non sa in quale nodo) e che è associata ad una certa chiave (nota). Il punto è quello di capire in **quale nodo è inserita questa informazione** e contattarlo.

P2P – Repository

Si inserisce un nodo speciale dove vengono memorizzati i messaggi di uno o più topic.

1. Il publisher associa una chiave a X e memorizza il messaggio nella repository.
2. Il subscriber trova la repository associata a X e si sottoscrive al topic.
3. Il subscriber riceverà direttamente dalla repository e non dal publisher.

Il problema ora è **come trovare la repository dato il topic**, per farlo ci sono due approcci:

- Approccio centralizzato, con coordinator.
- Approccio con flooding.

Ricerca repository con approccio centralizzato – Coordinator

Il coordinator è un nodo speciale che si occupa di trovare la repository di un certo topic. Nell'approccio P2P era il publisher che doveva trovare la repository, adesso non deve più farlo.

Il coordinator possiede una tabella di lookup (topic-repository).

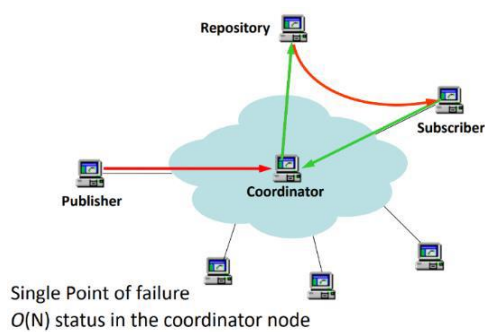
1. Il publisher invia messaggi di un topic X al coordinator.
2. Il coordinator assegna una chiave al topic e decide la repository.

L'identità della repository non è mai nota ai publishers.

3. Il subscriber richiede al coordinator l'accesso ai messaggi del topic.
4. Il coordinator la indica al subscriber.

L'identità della repository è nota ai subscribers.

5. **Repository e subscriber dialogano direttamente** senza il coordinator.



Ricerca repository con approccio decentralizzato – Flooding

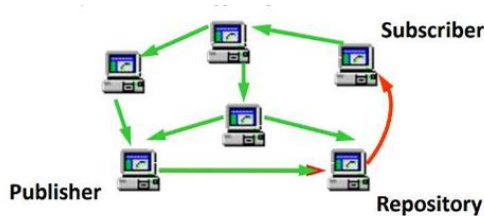
1. Quando un publisher pubblicizza un topic X, **decide la repository**.
2. La repository, dopo aver ricevuto una *advertise(X)* deve permettere ai subscriber di sottoscrivere e di distribuire messaggi con la stessa modalità dell'approccio con il coordinator.

Saranno i subscriber a dover fare fatica per trovare la repository → complessità sui subscriber.

3. **Flooding** → Il subscriber invia *subscribe(X)*, questo viene inoltrato ad ogni nodo nel radio range in modo che raggiunga il publisher o la repository.

4. Se il messaggio inviato dal subscriber raggiunge il publisher o il repository, essi invieranno una risposta al subscriber.

Per la risposta non serve il flooding perché il subscriber può indicare il proprio ID nel messaggio di iscrizione in modo da essere contattato direttamente dal publisher e/o dalla repository.



Tutti i problemi legati al flooding:

- Maggiore è la dimensione della rete, più tempo è necessario al flooding.
- Tempo di attesa sia per una risposta positiva, negativa o per una risposta che non arriverà mai.
- Il numero di messaggi elevato.

Quali sono i limiti dell'approccio centralizzato?

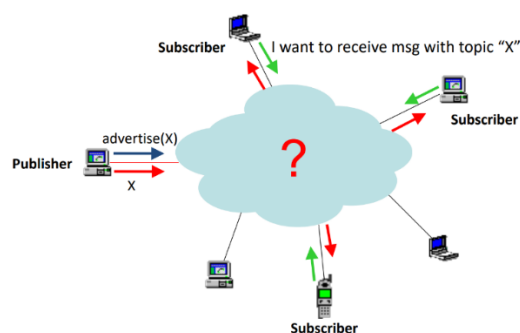
Il coordinatore non è paragonabile al server di prima perché il server memorizza i messaggi, mentre il coordinatore memorizza le associazioni chiave-repository.

- **Il coordinatore è un single-point-of-failure** → Risolvibile replicando.
- **Poco scalabile** → Risolvibile replicando e facendo load balancing.
- **Costo in termini di memorizzazione è $O(n)$** con n numero di publisher.

PS - Descrivi le due le fasi di funzionamento generali

Supponiamo un approccio topic-based.

1. **Advertising:** Il publisher fa advertising (X), con cui indica al rendez-vous che verrà inviato uno stream di messaggi del topic X.
2. **Subscribing:** Il subscriber, in maniera asincrona rispetto al publisher, indica al punto di rendez-vous con una subscribe (X) che vuole ricevere messaggi di X.



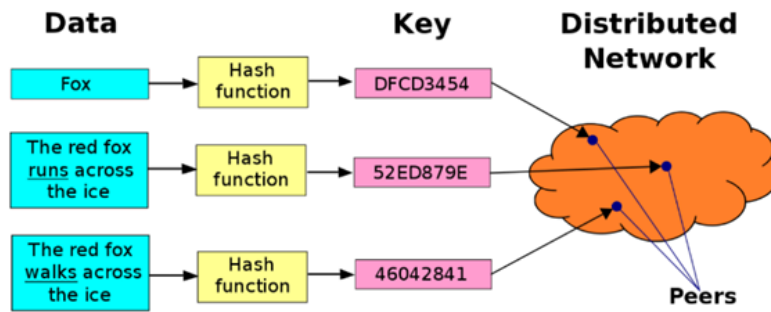
Possiamo quindi definire due fasi di funzionamento del sistema:

1. Le frecce **blu** e le frecce **verdi** rappresentano la fase di **advertising** e **subscribing**.
2. Le frecce **rosse** rappresentano la fase in cui i **messaggi** sono inviati ai subscriber.

DHT in generale

Per avere una **ricerca della repository efficiente** ma senza i problemi del coordinator e del flooding.

DHT → La responsabilità del mapping è distribuita tra i peers. Ogni peer memorizza un sottoinsieme della tabella.



Keyspace e mapping

Una funzione di hash trasforma le informazioni in un formato di **dimensione fissa** e l'univocità non è garantita.

Mapping → Operazione che associa una *key* al *value*.

In una DHT le chiavi sono distribuite su molti nodi.

- **Mapping uniforme** → i valori di output sono uniformemente distribuiti lungo il keyspace.

Per avere un mapping uniforme dobbiamo minimizzare le probabilità di collisione: in modo da evitare che particolari nodi presenti all'interno della rete siano soggetti a un carico maggiore.

- **Bassa probabilità di collisione** → deve essere raro che due valori abbiano la stessa chiave.

Per minimizzare le probabilità di collisione serve un **keyspace molto grande** (160 bit).

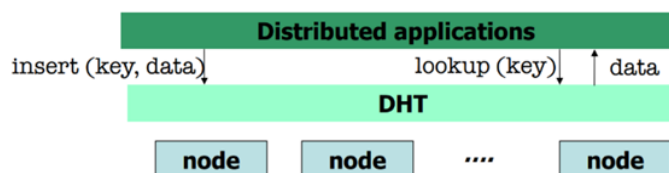
Caratteristiche DHT

Una DHT è caratterizzata dalle seguenti proprietà:

- **Autonomia e decentralizzazione** non esiste alcuna autorità centrale.
- **Elevata scalabilità**.
- **Tolleranza ai guasti**: Un sistema P2P è molto dinamico, la DHT deve essere tollerante ai guasti.
 - Se un nodo viene rimosso, le chiavi in quel nodo devono essere spostate in un altro nodo.
- **Bilanciamento del carico** le chiavi sono distribuite in maniera bilanciata (evitare colli di bottiglia) tra i nodi.
- **Anonimità**: la posizione fisica dei nodi non è nota agli altri nodi.

Le operazioni consentite in una DHT.

- **insert(key, data)** inserire un certo dato secondo una certa chiave.
- **lookup(key)** restituisce i dati associati a tale chiave, se presenti. Se non presenti, il lookup fallisce e la chiave non è memorizzata nel sistema.



L'applicazione che esegue le funzioni insert e lookup ha l'impressione di lavorare su un solo nodo.

Pastry - Come funziona, caratteristiche e funzionalità principali.

In Pastry viene implementata una DHT con paradigma P2P e rete con ridondanze.

- **Ogni nodo è caratterizzato da una tabella di routing**, costruita e riparata in maniera dinamica.
- **Non si usa flooding** ma un algoritmo di routing logaritmico basato su una topologia circolare.
- **Non ci sono SPoF**, ogni singolo nodo può abbandonare la rete senza provocare perdita.

- **Bilanciamento del carico:** dati e traffico distribuiti uniformemente tra i nodi.

Topologia circolare

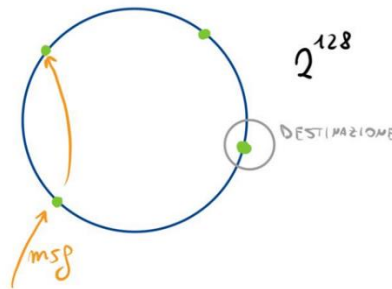
Nodi e chiavi sono organizzati su uno spazio che va da $[0, 2^b)$ incrementando il massimo di torna al primo.

Due nodi possono essere vicini perché hanno NodeID numericamente vicini, ma possono trovarsi distanti in internet.

NodeID

$NodeID = hash(IP)$, usare una funzione di hash su un keyspace grande (> 128 bit) garantisce:

- **Univocità.**
- **Mapping uniforme** → lookup efficiente.



Routing – Tabella di routing e leaf set

Schema di routing per attraversare il cerchio fino a trovare il nodo con NodeID numericamente più vicino alla chiave k .

Routing logaritmico → con N nodi → routing in $\log_{2^b}(N)$ step.

1. **Matching progressivo dei prefissi:** Si cerca il nodo più vicino, confrontando k con i NodeID dei nodi conosciuti.
2. **Salto lungo:** La **routing table** fa avanzare il messaggio in base al prefisso comune con salti di almeno una cifra alla volta nel prefisso.
 - a. Parità di numero di cifre uguali → RTT minimo.
3. **Salto corto:** Il **leaf set** interviene quando non ci sono più nodi con un prefisso migliore nella routing table, scegliendo il nodo numericamente più vicino.

Nel caso peggiore, il messaggio si sposta lentamente attraverso la Leaf Set, ma il percorso rimane logaritmico.

Leaf set			
	SMALLER	LARGER	
10233033	10233021	10233120	10233122
10233001	10233000	10233230	10233232

Routing table			
-0-2212102	1	-2-2301203	-3-1203203
0	1-1-301233	1-2-230203	1-3-021022
10-0-31203	10-1-32102	2	10-3-23302
102-0-0230	102-1-1302	102-2-2302	3
1023-0-322	1023-1-000	1023-2-121	3
10233-0-01	1	10233-2-32	
0		102331-2-0	
		2	

Neighborhood set			
13021022	10200230	11301233	31301233
02212102	22301203	31203203	33213321

Tolleranza ai guasti di Pastry – Leaf Set

Ogni nodo ha un “leaf set” che contiene i NodeID dei vicini (nodi con NodeID numericamente vicini al proprio).

Tipicamente il leaf set ha dimensione $l = 16$, quindi un nodo ha $\frac{l}{2}$ nodi numericamente vicini a destra ed $\frac{l}{2}$ a sinistra sull'anello.

Più capiente è il leaf set di ogni nodo e più affidabile diventa Pastry, ma diventa anche più lento.

Perché più affidabile? → **replica la chiave in uno dei nodi del leaf set**. Se il principale va offline, la chiave resta disponibile in un altro nodo della stessa “zona” della rete.

Ingresso nella rete – Neighborhood set

In questa tabella sono contenuti i nodi che sono fisicamente vicini: possono avere NodeID molto distanti, ma la distanza fisica è vicina perché magari appartengono alla stessa LAN.

Il Neighborhood set non viene usato per il routing:

- **Costruzione della Routing Table:** Quando un nodo entra nella rete, deve costruire la sua routing table scegliendo un nodo nella rete con cui può fare una comunicazione efficiente. Per questo motivo, sceglie nodi fisicamente vicini per ridurre la latenza.
- **Ripristino di un nodo guasto:** Se un nodo vicino fallisce, il NS può suggerire un nodo alternativo fisicamente vicino per rimpiazzarlo nella routing table.

Adesso dobbiamo capire cosa fare quando un nuovo nodo vuole unirsi alla rete Pastry ed è una cosa che succede spesso dato che Pastry è stato progettato per un ambiente P2P.

1. **Generazione NodeID:** Crea il suo NodeID tramite la funzione di hash.
2. **Contatto:** Dobbiamo contattare un nodo (A) nella rete Pastry (detto anche “nodo bootstrap”).

Data una lista di nodi della rete, si sceglie con minor RTT.

3. A controlla se il nuovo nodo è vicino ad esso:
 - a. Sì → lo inserisce nel proprio leaf set e informerà tutti i vicini → fine.
 - b. No → A instrada un messaggio di join (`lookup(X)`) per trovare il nodo più vicino a X .
4. **Inoltro del Join al nodo vicino a X :** Il messaggio viene inoltrato al nodo più vicino a X .

... Si attraversano vari nodi, fino a Z .

Alla ricezione della Join, ogni nodo attraversato invia una copia della propria tabella di routing a X .

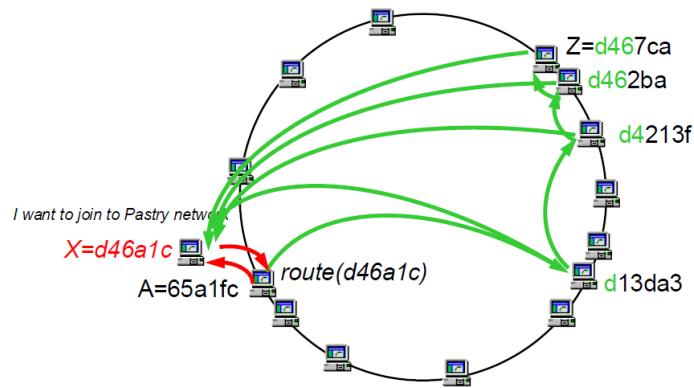
5. **Controllo NodeID:** Z riceve la richiesta di unirsi di X e fa lo stesso controllo.

Il controllo ha successo, dato che Z è il nodo più vicino a X , quest'ultimo viene inserito nel leaf set di Z .

6. **Riempimento delle strutture:** X può iniziare a riempire:
 - a. Leaf set → Copia quello di Z .
 - b. Tabella di routing → Si copiano i NodeID raccolti dai nodi che il messaggio di Join ha attraversato per raggiungere Z .

Supponendo di aver fatto un salto per ogni cifra: si copia la i° riga della tabella di routing dallo i° nodo incontrato lungo il percorso.

7. **Aggiornamento dei vicini:** X invia un messaggio a tutti i nodi che sono stati coinvolti.



Come funziona la Routing Table? Fanne un esempio.

$$Num_{Colonne} = 2^b$$

$$Num_{Righe} = \lceil \log_{2^b} N \rceil$$

In questo caso b non è il numero di bit dello ID, ma è il parametro “base”: indica **quanti bit** vengono trattati insieme come una **cifra** nella rappresentazione dei NodeID.

Noi tratteremo le cifre come numeri esadecimali $\rightarrow b = 4$ (ogni 4 bit è una cifra del prefisso).

Il **present Node ID** è il nodo che riceve la richiesta di lookup: trovare il value relativo ad una certa chiave k .

Ciascuna colonna nella n° riga si riferisce a un nodo il cui NodeID ha le prime n cifre (prefisso) uguali al present. La $(n + 1)^{\circ}$ cifra ha uno dei $2^b - 1$ possibili valori diversi dalla $(n + 1)^{\circ}$ cifra nel present (ci sono 2^b colonne).

Le cifre successive (da $n + 1$ in poi) sono arbitrarie, nel senso che dipendono dai nodi realmente presenti nella rete.

Supponiamo che il NodeID del present node sia 65a1xx...xx.

0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f
x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x
6	6	6	6	6	6	6	6	6	6	6	6	6	6	6	6
0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f
x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x
6	6	6	6	6	6	6	6	6	6	6	6	6	6	6	6
5	5	5	5	5	5	5	5	5	5	5	5	5	5	5	5
0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f
x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x
6	6	6	6	6	6	6	6	6	6	6	6	6	6	6	6
5	5	5	5	5	5	5	5	5	5	5	5	5	5	5	5
a	a	a	a	a	a	a	a	a	a	a	a	a	a	a	a
0	2	3	4	5	6	7	8	9	a	b	c	d	e	f	x
x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x

Perchè nella quarta riga abbiamo una cella vuota (/) nella seconda colonna?

Perché non possiamo avere un NodeID che inizia per 65a1 visto che condividerebbero le prime quattro cifre e non le prime tre.

Quindi i nodi con NodeID più vicini al nodo corrente sono rappresentati nelle ultime righe.

Come si individua la posizione di una chiave?

Chiavi e NodeID condividono lo stesso spazio $\rightarrow$ scegliamo il nodo con NodeID numericamente più vicino alla chiave.

Numericamente vicini → si sceglie il nodo con il prefisso comune più lungo, ovvero quello che condivide il maggior numero di cifre più significative.

Tra questi nodi (in caso di pareggio) viene scelto quello con la metrica di prossimità minore, ad esempio l'RTT.

Come si calcola il NodeID in Pastry?

$NodeID = hash(IP)$

Lookup in Pastry

Supponiamo di avere la chiave d46a1c memorizzata nel nodo d467ca.

1. Il nodo 65a1fc riceve route (d46a1c).

Compara quel valore col proprio NodeID e vede che la prima cifra è diversa (6).

2. Tramite la RT, il nodo legge la riga 0 e inoltra a d13da3.

Dato che il nodo è stato scelto nella prima riga, il salto compiuto è molto lungo.

3. Il nodo d13da3 riceve route (d46a1c).

Compara quel valore col NodeID e vede che la seconda cifra del prefisso è diversa (d1d4).

4. Tramite la RT, nodo d13da3 legge la riga 1 e inoltra a d462ba.

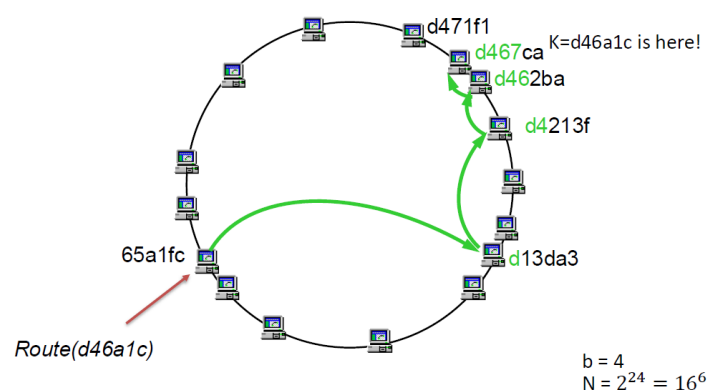
In questo caso abbiamo fatto un "salto bonus", poiché nella riga era presente un nodo con 3 cifre in comune.

5. Il nodo d462ba riceve route (d46a1c).

Compara quel valore col NodeID e vede che la quarta cifra del prefisso è diversa (d462).

6. Tramite la routing table, nodo d13da3 legge la riga 5 e inoltra a d467ca.
7. Il nodo d467ca riceve la richiesta route (d46a1c) e vede che la chiave l'ha memorizzata lui.
8. In questo caso siamo stati fortunati perché siamo riusciti a completare i salti senza usare il leaf set ma usando solo la routing table.

Abbiamo effettuato **4 salti sui 2^{24} possibili**.



Cosa succede al variare dei parametri "b" (base) ed "l"?

$$Num_{Colonne} = 2^b$$

$$Num_{Righe} = \lceil \log_{2^b} N \rceil$$

Valore di b	Pro	Contro
Più grande	Meno righe nella routing table → meno memoria e meno salti nel routing	Ogni riga ha molte colonne → serve più informazione iniziale per riempirla
Più piccolo	Meno colonne per riga → più facile da popolare con nodi conosciuti	Più righe → più hop nel routing

La dimensione totale del leaf set è l , quindi contiene $\frac{l}{2}$ nodi con ID più piccoli e $\frac{l}{2}$ con ID più grandi rispetto al nodo corrente. Un leaf set più grande:

- Consente al sistema di gestire meglio i guasti dei nodi vicini.
- Se la destinazione è vicina al nodo corrente, il leaf set può permettere un routing diretto senza consultare la routing table
- Implica un overhead maggiore nell'aggiornamento dello stesso.

Spiega la località in Pastry

In Pastry è presente una funzione che calcola una metrica di prossimità tra il nodo corrente e quello successivo (a cui inviare il messaggio), come l'RTT, che permette di scegliere il prossimo nodo.

Questo perché i nodeID numericamente vicini possono risultare geograficamente molto distanti tra loro.

Spiega la capacità di tolleranza alle disconnessioni

Grazie al **leaf set**, ogni volta che una chiave viene salvata in un nodo si ha ridondanza: il salvataggio della chiave può avvenire anche nei nodi del leaf set.

Dunque, con guasti simultanei dei nodi, la consegna finale è garantita, a meno che $\frac{l}{2}$ o più nodi adiacenti non falliscano simultaneamente.

I nodi nel leaf set si scambiano periodicamente **keep-alive**. Se un nodo non risponde per un periodo T , si presume che sia fallito.

Tutti i membri del set di leaf del nodo fallito vengono quindi avvisati e aggiornano i loro set di leaf.

Un nodo in fase di recupero (tornare nella rete) contatta i nodi nel suo ultimo set di leaf e ottiene i loro set di leaf correnti, aggiorna il proprio set e quindi notifica ai membri del suo nuovo set della sua presenza.

Scribe in generale

Sistema publish-subscribe costruito sopra Pastry.

Un nodo appartenente alla rete può creare un gruppo. Gli altri nodi possono: aderire al gruppo, inviare un messaggio multicast a tutti i membri del gruppo. Per quest'ultima cosa si richiede la creazione di un multicast tree.

Partiamo dal publisher, supponiamo sia topic-based, che funzioni deve avere nell'API di Scribe?

- `create(credentials, groupId)` crea un gruppo con groupId.

Le credenziali vengono utilizzate per il controllo degli accessi

- `join(credentials, groupId, messageHandler)` fa sì che il nodo locale si unisca al gruppo con groupId.

Tutti i messaggi multicast ricevuti successivamente per quel gruppo vengono passati al gestore dei messaggi specificato

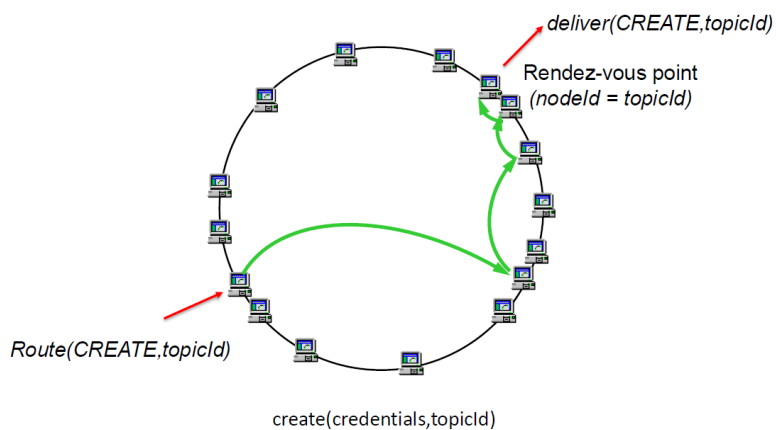
- `leave(credentials, groupId)` fa sì che il nodo locale lasci il gruppo con `groupId`
- `multicast(credentials, groupId, message)` fa sì che il messaggio venga trasmesso in multicast all'interno del gruppo con `groupId`

Join in Scribe

I nodi che vogliono unirsi al gruppo inviano una richiesta di join alla root.

Il root propaga i messaggi attraverso l'albero multicast, sfruttando Pastry. I nodi inoltrano i messaggi ai loro nodi sottostanti secondo l'albero multicast.

- Nodo A → ID: 1234
- Nodo B → ID: 4321
- Nodo C → ID: 8765
- Nodo D → ID: 2000
- Nodo E → ID: 7000



C vuole iscriversi al topic "meteo/roma".

1. Calcolo dell'hash del topic

```
groupId = hash("meteo/roma") = 4300
```

2. **Trovare il nodo root:** Pastry cerca il nodo più vicino a 4300, ovvero B (4321). Esso diventa il root del gruppo multicast per il topic "meteo/roma".

```
create(credentials=C , groupId=4300)
```

Questa chiamata forza la creazione del gruppo. B inizializza il gruppo multicast.

3. C si iscrive al topic.

```
join(credentials=C, groupId=4300, ... )
```

Routing di Pastry: richiesta inoltrata lungo $C \rightarrow E \rightarrow B$.

4. **Costruzione dell'albero multicast:**

- a. E riceve la join da C e registra C come suo "figlio" nell'albero multicast.
- b. B riceve la join da E e registra E come suo "figlio".
- c. C riceve una conferma: è ora parte del gruppo multicast.



5. Un altro subscriber A si unisce allo stesso topic.
 - a. join inoltrata lungo A → D → B.
 - b. Lungo il cammino, D e B aggiornano l'albero.

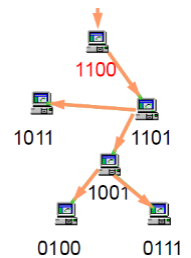
```
join(credentials=A, groupId=4300, ...)
```

6. D invia un messaggio sul topic "meteo/roma":
 - a. Il messaggio va a B (root).
 - b. D lo inoltra ad A.
 - c. B lo inoltra a E.
 - d. E lo inoltra a C

```
multicast(credentials=D, groupId=4300, message="Piove a Roma!")
```

7. A non vuole più ricevere notifiche.
 - a. Il messaggio va a B (root).
 - b. B inoltra a tutti.
 - c. Tutti i nodi nel percorso aggiornano l'albero rimuovendo A.

```
leave(credentials=A, groupId=4300)
```



Lookup in Scribe

Trovare la radice del gruppo associato a un topic. Responsabile della gestione dell'albero multicast per quel topic.

```
create(credentials=C , groupId=4300)
```

```
join(credentials=C , groupId=4300, ... )
```

1. **Trovare il nodo root:** Pastry cerca il nodo più vicino al groupId, ovvero B (4321). Esso diventa il root del gruppo multicast.
2. **Costruzione dell'albero multicast,** tramite il routing di Pastry la richiesta viene inoltrata.
 - a. E riceve la join da C e registra C come suo "figlio" nell'albero multicast.
 - b. B riceve la join da E e registra E come suo "figlio".
 - c. C riceve una conferma: è ora parte del gruppo multicast.

Human As A Sensor

HaS in generale

Usare gli esseri umani come dei sensori, in grado di fare delle osservazioni e di riportare ciò che vedono e pensano.

Crowdsensing

Sfruttare una **grande mole di persone** che fornisce informazioni tramite una **piattaforma social** in **tempo reale**.

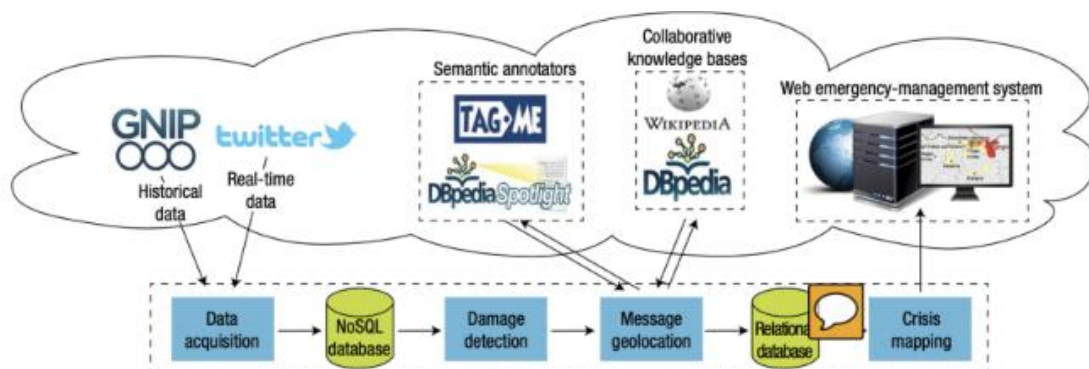
- **Crowdsensing partecipativo**: utenti **consapevoli** che stanno condividendo informazioni, decidono volontariamente.
- **Crowdsensing opportunistico**: utenti **non consapevoli**, oppure non coinvolti direttamente nella collezione dei dati.

I social sono un esempio improprio, perché gli utenti pubblicano informazioni volontariamente ma non sono consapevoli dell'uso che ne verrà fatto.

Esempio di crowdsensing

Abbiamo tanti testimoni oculari con a disposizione la possibilità di comunicare.

In queste situazioni (eventi drammatici), l'essere umano ha l'esigenza di comunicare in modo oggettivo cosa succede e questa cosa ci aiuta moltissimo, perché rende l'umano un sensore più affidabile.



1. **Data acquisition**: Acquisizione dei dati storici per mezzo di dump e di dati recenti dai social network (API di Twitter).
2. **Storing**: Le Informazioni sono memorizzate all'interno di un database NoSQL.

Quindi i dati "raw" vengono inseriti in un database molto veloce e adatto ai Big Data.

3. **Damage Detection**: Le informazioni sono passate al setaccio per individuare danni.
4. **Message geolocation**: Se si individua un' emergenza si cerca di ottenere più informazioni.

Si fa affidamento anche a siti come Wikipedia, i cui collaboratori aggiornano i contenuti in tempo reale.

5. **Storing**: I risultati di questa elaborazione vengono memorizzati in un database relazionale e possono essere utilizzati dall'unità di crisi per assumere decisioni.

I dati elaborati vengono inseriti in un database più complesso ma più adatto ad elaborazioni più profonde.

6. **Crisis Mapping**.

Nello HaS, quali sono i due problemi principali?

*Model human participants as sources of **unknown reliability** generating **binary measurements of uncertain provenance**.*

Affidabilità Sconosciuta

L'affidabilità di un sensore fisico è nota, mentre quella dei sensori umani è sconosciuta.

Reliability → probabilità che l'utente esprima osservazioni vere, ovvero osservazioni che contribuiscono a una corretta rappresentazione dello stato del mondo reale.

Provenienza Incerta

I sensori fisici sono caratterizzati da rumore, stessa cosa per quelli umani.

Le persone (e le loro osservazioni) possono essere influenzate dalle osservazioni fatte dalle altre persone.

Possiamo quindi avere poche informazioni originali e tante informazioni non originali che potrebbero riportare informazioni sbagliate o distorte.

Come si definisce un claim e come si ottengono partendo da un social?

Token

Parola → Token.

Dato un set di tweet (osservazioni), viene individuato uno spazio di token, la cui dimensione corrisponde al **numero di parole diverse nel set di tweet**.

1 - Data collection

Raccolta dei dati e dipende fortemente dal tipo di applicazione in esame (usiamo un social come Twitter). Le sue API permettono di estrarre i dati desiderati ponendo alcuni filtri: keyword particolari, regione geografica.

2 - Data structuring

I dati devono essere organizzati in modo tale da poter essere analizzabili.

Utilizziamo una funzione di tokenizzazione su ogni tweet: ad ogni tweet associamo un vettore la cui dimensione è il numero di token nello spazio dei token. Ogni elemento restituisce il numero di occorrenze del relativo token nel tweet.

3 - Comparazione

Calcolare una distanza tra due vettori per vedere quanto sono simili due osservazioni (da 0 a 1).

La funzione di similarità più utilizzata è quella basata sul coseno, se due vettori sono simili allora avranno un angolo tale che il coseno sia vicino a 1.

$$\text{Similarity} = \cos \theta = \frac{A \cdot B}{\|A\| * \|B\|} \in [-1, 1]$$

```
doc_1 = "Data is the oil of the digital economy"
doc_2 = "Data is a new oil"
```

```
# Vector representation of the document
doc_1_vector = [1, 1, 1, 1, 0, 1, 1, 2]
doc_2_vector = [1, 0, 0, 1, 1, 0, 1, 0]
```

	data	digital	economy	is	new	of	oil	the
doc_1	1	1	1	1	0	1	1	2
doc_2	1	0	0	1	1	0	1	0

$$\begin{aligned}
 A \cdot B &= \sum_{i=1}^n A_i B_i \\
 &= (1 * 1) + (1 * 0) + (1 * 0) + (1 * 1) + (0 * 1) + (1 * 0) + (1 * 1) + (2 * 0) \\
 &= 3
 \end{aligned}$$

$$\sqrt{\sum_{i=1}^n A_i^2} = \sqrt{1+1+1+1+0+1+1+4} = \sqrt{10}$$

$$\sqrt{\sum_{i=1}^n B_i^2} = \sqrt{1+0+0+1+1+0+1+0} = \sqrt{4}$$

$$\text{Similarity} = \frac{A * B}{||A|| * ||B||} = \frac{3}{\sqrt{4} * \sqrt{10}} = 0,474$$

4 - Individuazione dei claim

Costruiamo un grafo dove:

- I nodi sono le osservazioni individuali (tweet);
- Gli archi sono pesati, con peso pari alla somiglianza tra i due nodi.

Più è alta la similarità, più corto sarà l'arco e più vicine saranno le due osservazioni, formando dei clusters. **Ciascun cluster ottenuto è chiamato claim**, cioè un pezzo di informazione riportato da una molteplicità di fonti.

5 - Individuazione relazioni tra sorgente e claim

Costruiamo il **Source-Claim graph (SC)**, rappresenta le relazioni tra sorgenti e claim.

- Ogni fonte S_i (utente) è connessa a tutte le claim fatte da essa.
- Ogni claim C_j è connessa a tutte le fonti che l'hanno esposta in qualche maniera.
- Si pone un arco $S_i C_j = 1$ se la fonte ha effettivamente esposto il claim.

SC permette di avere una visione quantitativa del supporto per ogni claim, ma non segnala il rapporto tra le fonti (necessaria per poter mitigare la provenienza incerta).

6 Social Dissemination

Il **Social Dissemination (SD)**, stima come le informazioni potrebbero propagarsi da un utente a un altro.

Le reti generate in questo modo sono dette **Epidemic Cascade Network (EC)**: ogni osservazione è modellata come una cascata. In particolare, descrive il tempo di contagio.

Il "tempo di contagio" permette di analizzare:

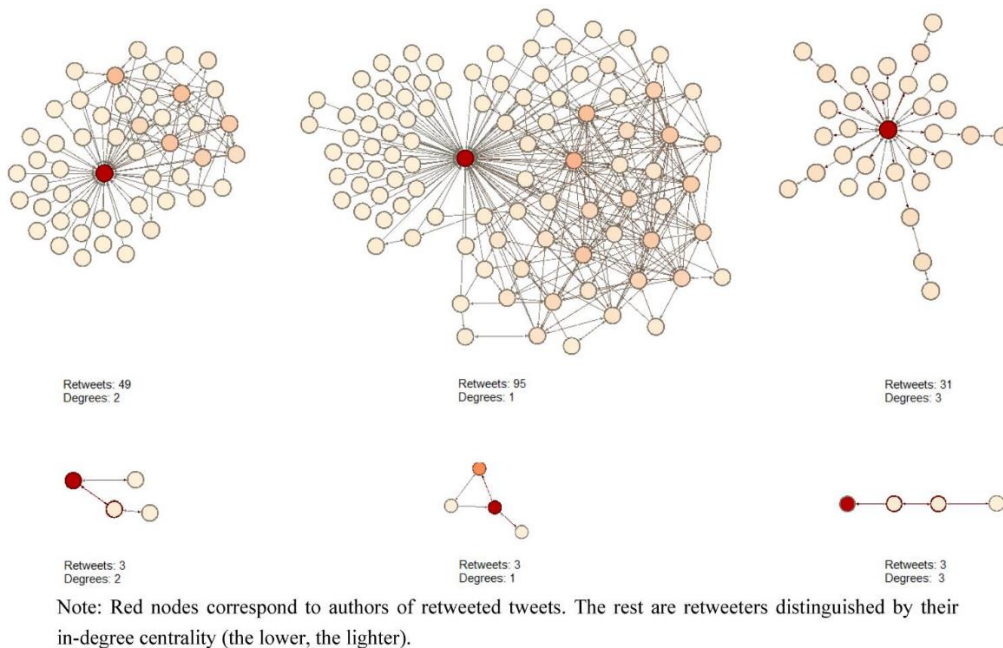
- Chi è il primo ad aver affermato un certo "claim" e chi lo ha affermato successivamente.

- La velocità con cui il claim si è diffuso sul social.
- Dove è stato affermato il claim all'interno del social (gruppi/pagine).

Per Twitter sono stati definiti questi grafici specifici (di fatto modellano il social network).

- **Follower-Followee network (FF):** Un arco $S_i S_k$ esiste se la fonte S_k è un follower della fonte S_i
- **Re-Tweeting network (RT):** Un arco $S_i S_k$ esiste se la fonte S_k ha ritwittato un messaggio dalla fonte S_i
- **Combined network (RT+FF):** Un arco $S_i S_k$ esiste se la fonte S_k ha ritwittato un messaggio dalla fonte S_i o se è un follower di S_i .

In questa figura notiamo degli esempi di grafi di relazione costruiti tramite i retweet.



Parla dell'MLE problem e di θ , in particolare di come si trova (in generale le variabili del problema)

Metodologia

1. Si sceglie la tipologia di evento di interesse;
2. Si individua tutto ciò che permette di identificare i tweet come relativi alla tipologia di evento scelta (keywords, posizione geografica, etc...)
3. Si estraggono i messaggi utilizzando le API Si memorizzano i tweet raccolti in database.
4. Si applicano tecniche di clustering per trovare i claims da analizzare.
5. SC ed SD.
6. MLE.

La estimation theory serve per decretare, dati SC e SD, la credibilità di un certo claim.

Il metodo più semplice è il **voting**, i claim che hanno un numero maggiore di voti: per ogni C_j , dove $1 \leq j \leq N$, contiamo tutti gli S_i , dove $S_i C_j = 1$.

Questa soluzione è subottimale, perché:

- **Sorgenti differenti hanno diverso grado di credibilità**, quindi il loro "voto" ha peso differente.
- **Le sorgenti potrebbero non essere indipendenti**, quindi possono influenzarsi a vicenda.

LE – Likelihood Estimation

Si tratta di calcolare la probabilità di correttezza di un'osservazione prendendo in considerazione l'affidabilità sconosciuta delle fonti e della loro provenienza incerta.

Dati i grafi SD e SC, quale è la probabilità che un'osservazione C_j sia vera per ogni j ?

$$\forall j \in [1, N] : P(C_j = 1 \mid SC, SD)$$

Expectation Maximization (risolve l'affidabilità sconosciuta)

Sia m il numero totale di fonti da cui riceviamo dati. Descriviamo ogni fonte con due parametri (sconosciuti all'inizio):

- **Probabilità di veri positivi** $a_i = P(S_i C_j \mid C_j = 1)$ la probabilità di affermare cose vere.
- **Probabilità di falsi positivi** $b_i = P(S_i C_j \mid C_j = 0)$ la probabilità di affermare cose false.

Sia il rateo di osservazioni corrette del sistema: $d = P(C_j = 1)$

Sia $\theta = [a_1, a_2, \dots, a_m, b_1, b_2, \dots, b_m, d]$ il vettore dei **parametri sconosciuti**.

MLE - Maximum Likelihood Estimation (risolve la provenienza incerta)

Trovare i valori di θ tali che massimizza la probabilità di osservazione di SC, dato il social SD.

$$P(SC \mid SD, \theta)$$

Supponendo che i claim siano indipendenti tra di loro, applichiamo il **th della probabilità totale**:

$$p(SC \mid SD, \theta) = \sum_z p(SC, z \mid SD, \theta)$$

Dove z è un vettore i cui elementi $z_j = 1$ se C_j è vera.

Se individuiamo i valori di z e θ , abbiamo finito: conosciamo l'affidabilità delle fonti (θ) e se i claim sono veri (z). La soluzione si trova tramite algoritmi appartenenti all'ottimizzazione (the iterative Expectation Maximization Algorithm).

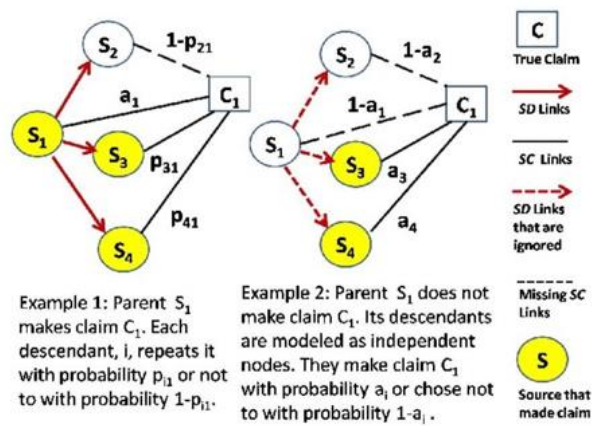
Dobbiamo ancora risolvere il problema della provenienza incerta, considerando il grafo SD:

Se il padre S_k ha fatto la stessa claim del figlio S_i , allora il figlio ripete il procedimento con una certa probabilità di ripetizione p_{ik} .

$$p_{ik} = \frac{\text{Numero di volte che } S_i \text{ e } S_k \text{ hanno fatto la stessa osservazione}}{\text{Numero di osservazioni di } S_k}$$

Se $p_{ik} = 0$ allora le due fonti sono indipendenti.

Se il padre sostiene un claim e il figlio no, allora si suppone che il figlio agisca in maniera indipendente con probabilità $1 - p_{ik}$.



Si parla di EM se si intende l'algoritmo precedente tranne la parte finale sul SD: questo significa non poter distinguere sorgenti indipendenti da sorgenti affidabili.

- Regular-EM, tutte le fonti sono considerate osservatori indipendenti
- Regular-EM-AD, le fonti dipendenti da altre fonti sono rimosse sfruttando approcci euristici.

Cima

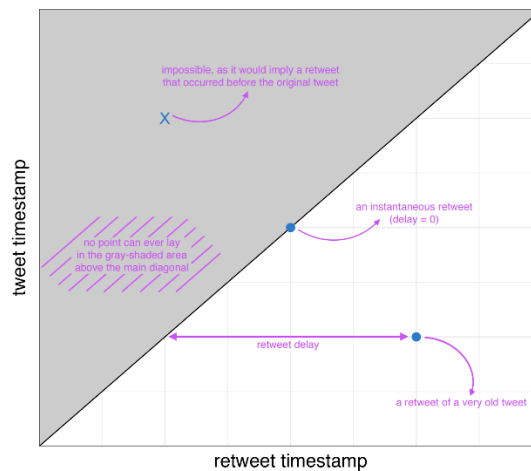
Bot detection in generale

Botnets → insieme di bot con lo stesso preciso comportamento. Le botnet si possono riconoscere attraverso il “coordinated behaviour”, ovvero comportamenti/linee di pensiero simili/uguali.

Esistono poi i Social bot, ovvero account controllati dal software che interagiscono in modo più o meno automatico.

Spesso i bot non vengono riconosciuti da parte degli umani. Per riconoscerli, gli algoritmi di AI cercano i pattern, dei comportamenti ricorrenti nel tempo.

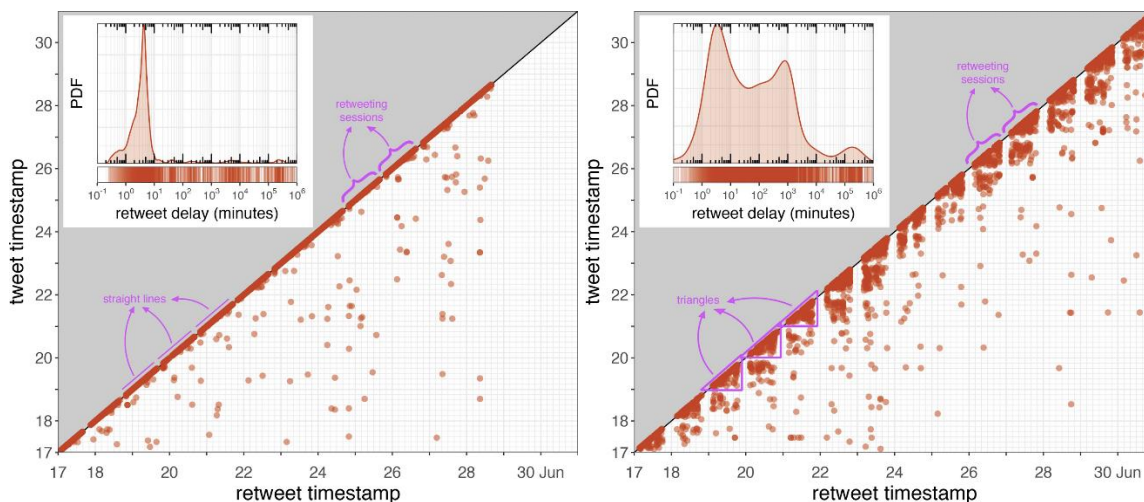
Temporal behaviour



Ogni punto rappresenta un evento di retweet, con:

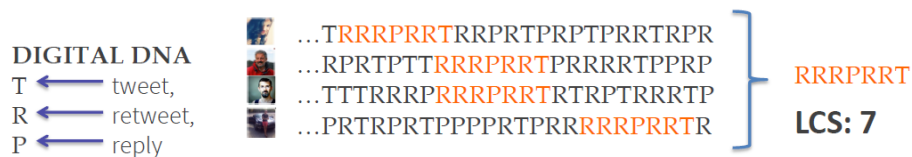
- La posizione sull'asse Y che mostra quando è stato postato il tweet originale.
- La posizione sull'asse X che mostra quando è stato fatto il retweet.

Comportamenti sospetti: punti troppo vicini o troppo lontani dalla retta → molti utenti hanno retweettato lo stesso contenuto allo stesso momento del tweet originale o che lo hanno retweettato dopo molti giorni.



DNA Digitale

Il DNA vero è composto da comportamenti attuati in un social. Possiamo confrontare gruppi di utenti per rilevare delle botnet. **I bot avranno sequenze simili.** Viene calcolata la LCS (Longest Common String).



Network-based approach

Undirected/directed
Graph

	0	0	0	1	1	1	0	0	0	0
	0	0	0	1	1	1	0	0	0	0
	0	0	0	1	1	1	0	0	0	1
	0	0	0	0	0	0	0	0	0	0
	0	0	0	0	0	0	0	0	0	0
	0	0	0	0	0	0	0	0	0	0
	0	0	0	0	1	0	0	0	1	0
	0	0	0	0	0	0	0	0	1	1
	0	0	0	0	0	0	0	0	0	0
	0	0	0	0	0	0	0	0	0	0

Nella matrice di adiacenza (riga = account A, colonna = account B, valore 1 se A interagisce con B), i bot organizzati appaiono come blocchi molto connessi, perché si seguono e retwittano tra loro in modo coordinato e ripetuto.

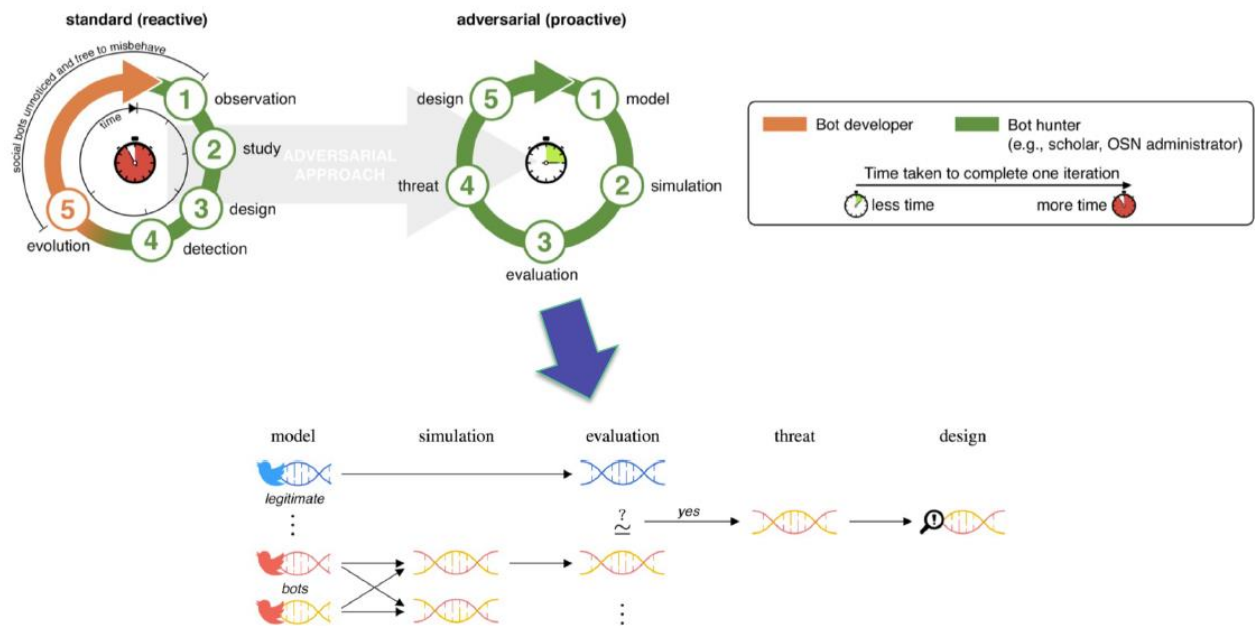
- **Utenti reali** → hanno connessioni sparse e varie.
- **Botnet** → mostrano sottomatrici densamente popolate.

Advanced Modern Approaches

Tutte queste tecniche non sono sufficienti, perché i bot migliorano e devono farlo anche le tecniche.

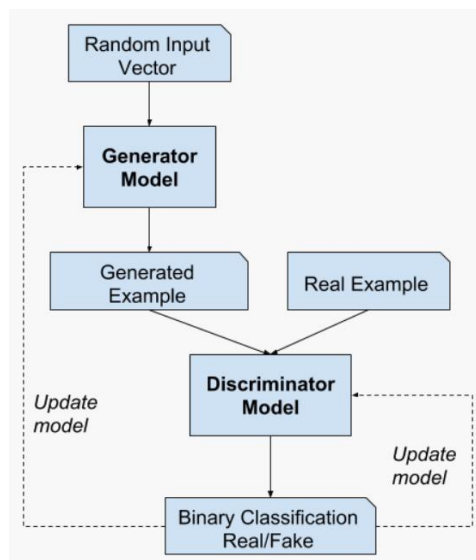
Le tecniche di ML sono state progettate per ambienti stazionari e benigni. Quando queste ipotesi vengono violate, le tecniche iniziano a commettere grandi errori.

Per contrastare ciò è possibile utilizzare un **approccio proattivo**: attacchiamo il nostro sistema e troviamo una contromisura, in questo modo il sistema è in grado di riconoscere una buona parte degli attacchi, anche se non tutti.



Gli approcci proattivi si basano sull'uso delle GAN (Generative Adversarial Nets): uno scenario di teoria dei giochi in cui il generatore compete contro il discriminatore con lo scopo di addestrare il modello generativo.

Il generatore genera campioni plausibili (falsi) e il discriminatore distingue. L'addestramento si interrompe quando il discriminatore non riesce a distinguere i campioni reali da quelli falsi.



Un creatore di bot compete con un rilevatore di bot, obiettivo: addestrare un rilevatore di bot migliore (più robusto).

Community detection in generale

Identificano particolari pattern (topologie mesoscale) all'interno di reti complesse.

Mesoscale → gruppi di nodi che mostrano **pattern strutturali ricorrenti**.

Non esiste una vera definizione di comunità: "Un insieme di nodi più strettamente connessi tra di loro che con nodi appartenenti ad altri insiemi."

Simple Reward Function

- "Prendiamo" $1 - \gamma$ per ogni edge nel cluster.
- "Paghiamo" γ per ogni edge mancato.

La soluzione ottimale ha il guadagno (reward) più alto:

- H : reward;
- e_c : edge nel cluster;
- n_c : nodi nel cluster.

$$H = \sum_c (e_c - \gamma \binom{n_c}{2})$$

Se $\gamma = 0,5$:

con edge attesi: $N \times \frac{N-1}{2}$

$$n_c = 4 \rightarrow \text{edge attesi} = 4 \times \frac{3}{2} = 6$$

$$n_c = 3 \rightarrow \text{edge attesi} = 3 \times \frac{2}{2} = 3$$

$$n_c = 5 \rightarrow \text{edge attesi} = 5 \times \frac{4}{2} = 10$$

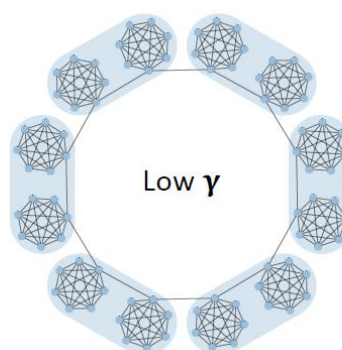
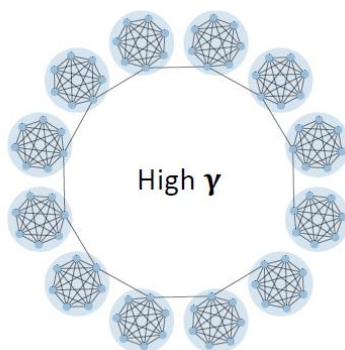
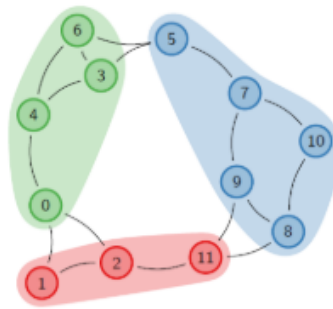
$$4 \text{ edges, } 2 \text{ missing} \rightarrow 4 \times 0,5 - 2 \times 0,5 = 1$$

Dove il primo 0,5 è $\gamma - 1$, mentre il secondo è γ

$$2 \text{ edges, } 1 \text{ missing} \rightarrow 2 \times 0,5 - 1 \times 0,5 = 0,5$$

$$5 \text{ edges, } 5 \text{ missing} \rightarrow 5 \times 0,5 - 5 \times 0,5 = 0$$

$$H = 1 + 0,5 + 0 = 1,5$$



Funzione di Modularità

Misura la densità di una comunità, **la soluzione ottimale ha la modularità più elevata.**

L'algoritmo di modularità è **greedy**: vengono effettuati molti tentativi e viene selezionata la funzione con reward $Q \in [-1; 1]$ maggiore.

Q indica quanto la densità dei link all'interno delle community sia superiore a quella attesa in una rete casuale.

Per i grafi diretti si usa la seguente formula di reward:

$$Q = \frac{1}{2m} \sum_{ij} \left[A_{ij} - \frac{k_i k_j}{2m} \right] \delta(c_i, c_j)$$

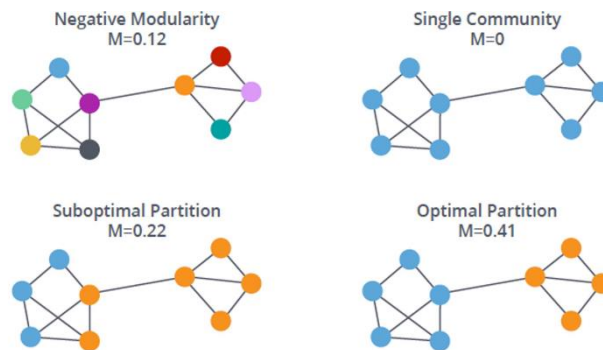
Normalization Factor.
 m : number of edges

Null Model expected density
($k_i = k_j$ if undirected)

1 if i, j in same community,
0 otherwise

Per i grafi non diretti invece si usa:

$$Q = \frac{1}{m} \sum_c \left(e_c - \frac{1}{2} \frac{K_c^2}{2m} \right)$$



Algoritmo di Louvain

Algoritmo **greedy** per trovare una **partizione in community** che **massimizza la modularità**:

Quanto i nodi sono più connessi tra loro **all'interno** delle comunità rispetto a quanto ci si aspetterebbe in una rete casuale.

Suddividere i nodi in **community**, cioè gruppi di nodi **densamente connessi tra loro** e **poco connessi con il resto della rete**.

1. **Inizializzazione**: ogni nodo viene assegnato ad una comunità.
2. **Fase 1 – Spostamento dei nodi**: uno alla volta, ogni nodo viene spostato nella comunità adiacente che garantisce il maggiore incremento di Q .

Ripetere la fase 1 finché non viene spostato alcun nodo.

3. **Fase 2 - Aggregazione**: Si costruisce un nuovo grafo dove ogni community diventa un super-nodo e i pesi degli archi tra super-nodi sono la somma dei pesi degli archi originali tra le community. Gli **archi interni** a una community diventano dei **loop** sul super-nodo.
4. Si alternano fase 1 e 2 finché la modularità **non migliora più**.

Supponendo di usare una funzione di reward con $\gamma = 0,5$ invece della modularità.

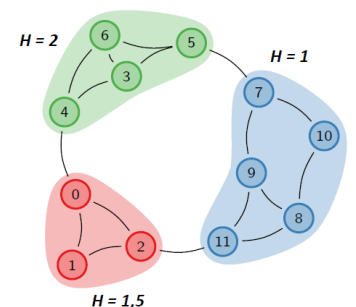
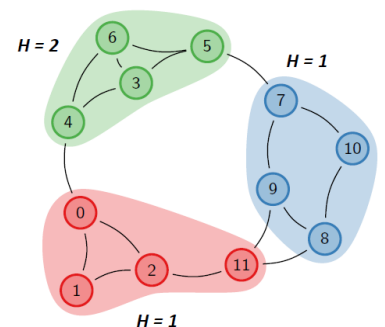
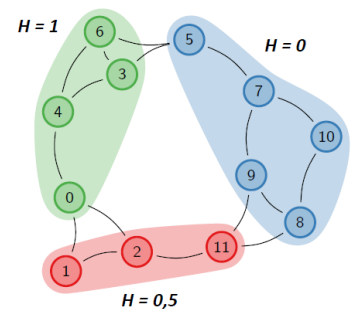
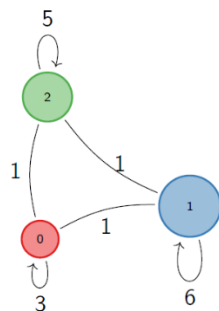
Inizialmente $H = 1,5$

- Spostare 0 nel **rosso**? Sì, due edge guadagnati, uno perso $\rightarrow H = 2,5$
- Spostare 1 nel **verde**? NO, ne guadagniamo uno e ne perdiamo uno.
- Spostare 2 nel **verde**? NO, perdiamo 3 edge.
- Spostare 3 nel **verde**? NO, ne guadagniamo uno e ne perdiamo uno.
- Spostare 4 nel **rosso**? NO, ne guadagniamo uno e ne perdiamo uno.
- Spostare 5 nel **verde**? SÌ, due edge guadagnati, uno perso $\rightarrow H = 4$
- Spostare 6 nel **blu**? NO.
- Spostare 7 nel **verde**? NO, ne guadagniamo uno e ne perdiamo due.
- Spostare 8? NO
- Spostare 9? NO
- Spostare 10? NO
- Spostare 11? SÌ, due edge guadagnati, uno perso $\rightarrow H = 4,5$

Ciclo terminato, *alcuni cluster sono stati modificati*? SÌ $\rightarrow$ riavvia il ciclo.

Ciclo terminato, *alcuni cluster sono stati modificati*? NO $\rightarrow$ aggregazione.

I numeri nell'arco entrante nello stesso nodo indicano il numero di archi nel cluster, i numeri negli archi tra i cluster indicano il numero di archi tra i cluster.



Algoritmo di Leiden

L'algoritmo di Louvain ha alcuni problemi noti:

- Può generare community disconnesse internamente (con sottogruppi isolati).
- Può bloccarsi in partizioni subottimali (locali ma non globali).
- Non garantisce miglioramenti continui durante l'ottimizzazione.

L'algoritmo di Leiden è un'evoluzione dell'algoritmo di Louvain.

1. **Inizializzazione:** ogni nodo nella rete viene assegnato alla propria comunità e contrassegnato come instabile.

Instabile significa che potrebbe migliorare la modularità spostandosi.

2. **Fase 1 – Spostamento dei nodi:** I nodi instabili vengono spostati tra le community adiacenti per aumentare Q . I nodi diventano stabili quando non migliorano più la modularità in nessuna community vicina.
3. Si continua finché tutti i nodi sono stabili.

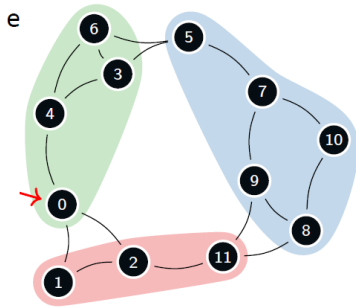
Ripeti la fase 1 finché non viene spostato alcun nodo. Dopo la fase 1, alcune community potrebbero non essere connesse internamente.

Leiden controlla ogni community: Se è disconnessa, viene scomposta in sottocommunity connesse.

4. **Fase 2 – Affinamento:** per suddividerle in sottocommunity connesse.
5. **Fase 3 - Aggregazione:** aggrega i cluster come nell'algoritmo di Louvain, tenendo in considerazione l'output di affinamento.

Si ripete tutto l'algoritmo (fasi 1, 2, 3) sul grafo aggregato. Le fasi 1, 2 e 3 vengono ripetute finché la modularità non viene massimizzata.

Fase 1



Supponendo di usare come ricompensa la semplice funzione di ricompensa con $\gamma = 0,5$ invece della modularità, quali sono le comunità di Leiden su questo grafico dopo la fase 1?

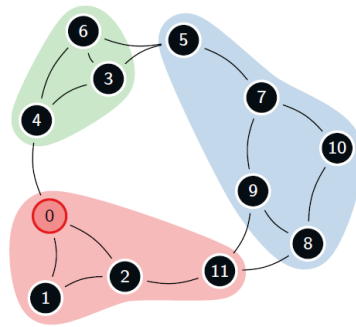
Inizializzazione: tutti i nodi sono instabili.

Spostare 0? SI → Segnare 0 come stabile e i vicini come instabili.

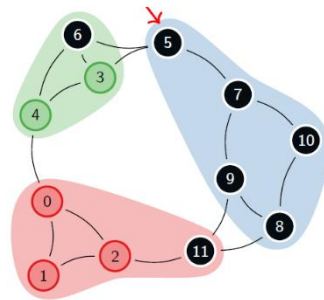
Spostare 1? NO → STABILE

...

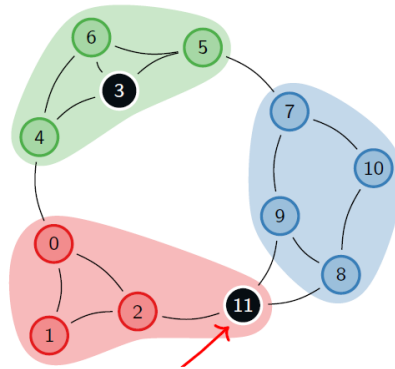
Spostare 4? NO → STABILE



Spostare 5? SI → Segnare 5 come stabile e i vicini come instabili: 6-7 sono ancora instabili, 3 diventa instabile.



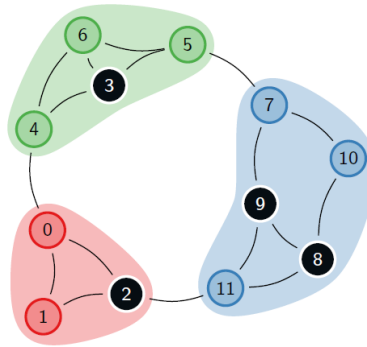
Spostare 6? NO => STABILE
 Spostare 7? NO => STABILE
 Spostare 8? NO => STABILE
 Spostare 9? NO => STABILE
 Spostare 10? NO => STABILE



Spostare 11? SI → Segnare 11 come stabile e i vicini instabili → 2-8-9 diventano instabili.

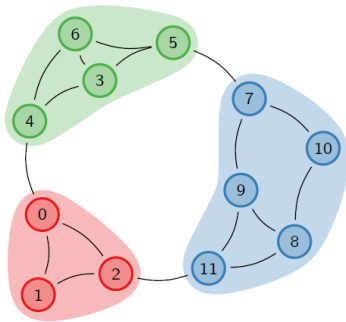
Ciclo terminato, alcuni cluster sono stati modificati? SI, riavvia il ciclo.

Spostare 2? NO => STABILE
 Spostare 3? NO => STABILE
 Spostare 8? NO => STABILE
 Spostare 9? NO => STABILE



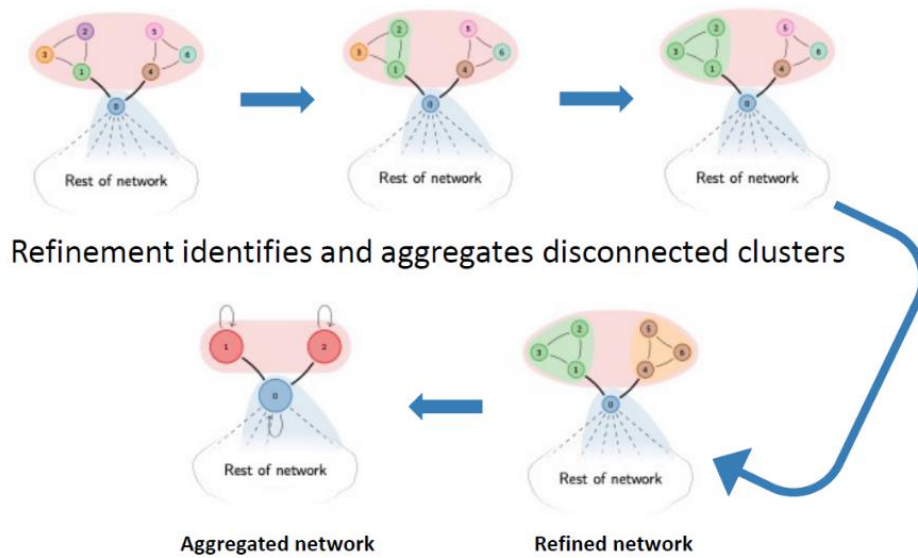
Ciclo terminato, alcuni cluster sono stati modificati? NO

Avviare il raffinamento e l'aggregazione.



Fase 2-3 - Raffinazione e aggregazione

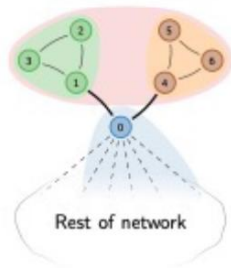
La raffinazione identifica e aggrega i cluster disconnessi.



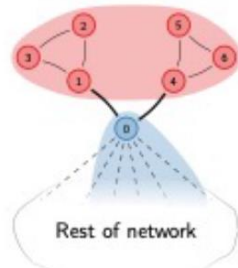
Leiden vs Louvain

Leiden è più veloce (controlla solo i nodi instabili) ma più complesso.

Il raffinamento gestisce meglio i nodi disconnessi, mentre Louvain mette tutti i nodi disconnessi in un cluster unico.



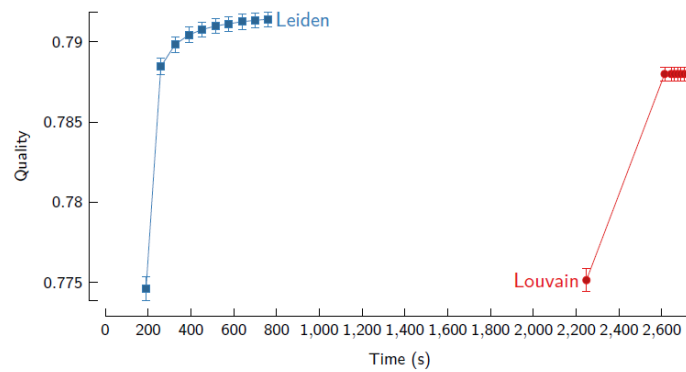
Leiden refinement



Louvain management

Leiden è migliore su reti di grandi dimensioni (potrebbe funzionare velocemente su reti con miliardi di link).

Leiden ha una prima iterazione molto più rapida e dopo l'iterazione stabile ottiene una qualità migliore rispetto a Louvain.



Vecchio

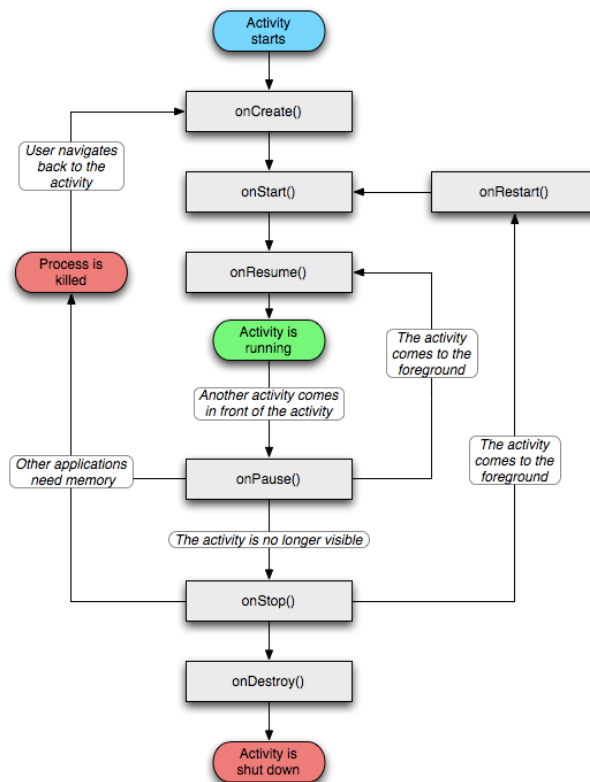
Android

Ciclo di vita di una attività

Un'attività è una singola schermata con una UI. Il ciclo di vita è una sequenza di **callback** chiamate da Android per notificare **stati** in cui l'attività entra o esce.

- **Visibile** → L'utente può vedere (anche parzialmente) l'interfaccia dell'attività sullo schermo.
- **Primo piano** → **attivamente in uso**, completamente in vista e **interattiva**. Se in primo piano è anche visibile ma non viceversa.

Situazione	Visibile?	In primo piano?
L'utente usa l'app normalmente (UI attiva e interattiva)	✓ SI	✓ SI
Una finestra di dialogo (es. AlertDialog) copre la UI	✓ SI	✗ No (onPause)
L'utente preme il tasto Home e torna alla Home	✗ No	✗ No
Una chiamata in arrivo oscura tutta l'UI	✗ No	✗ No



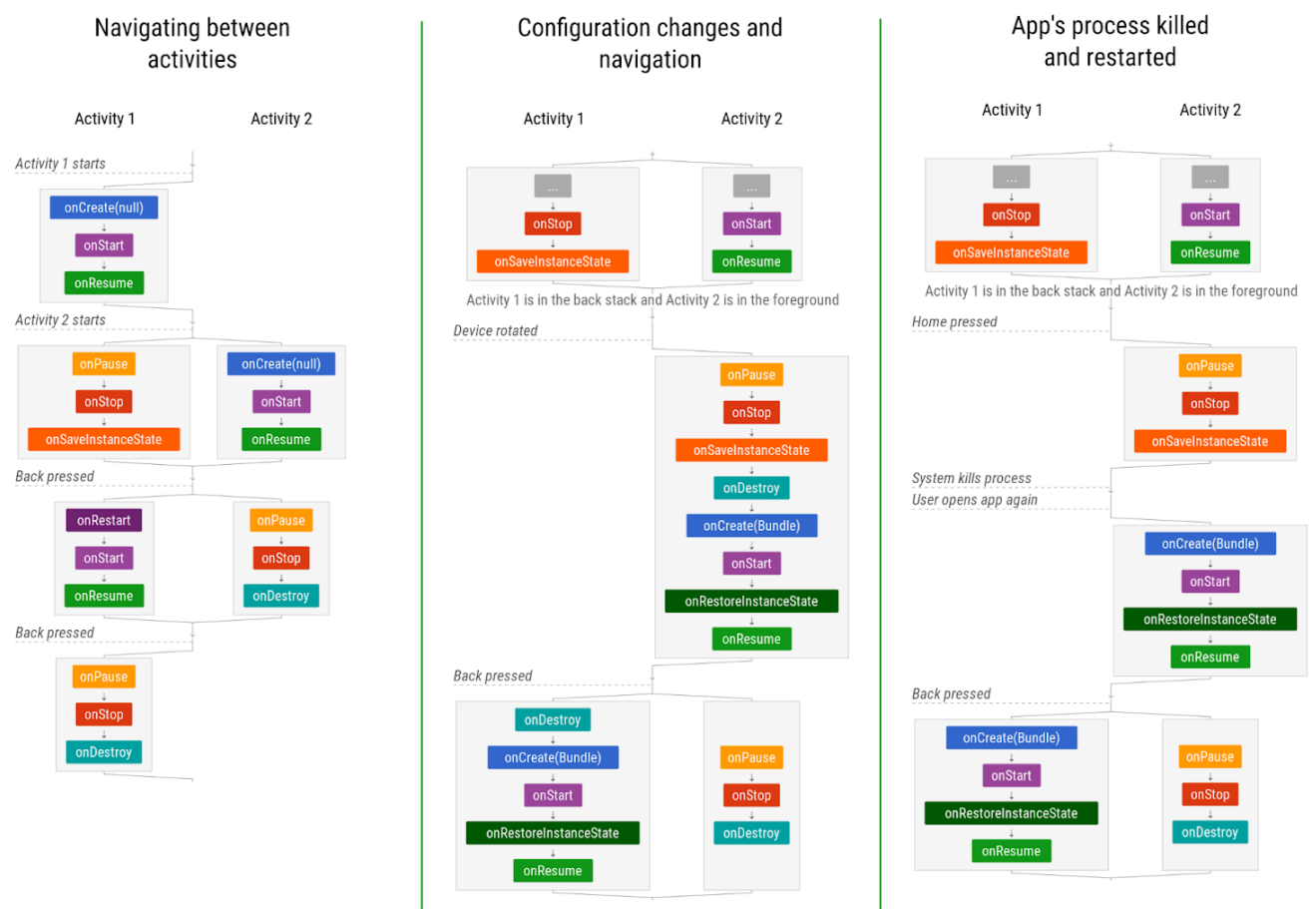
1. `onCreate()` → Viene **creata**. Qui si inizializza tutto: layout, dati, listener, ecc.
2. `onStart()` → **Visibile** ma non in primo piano.
 - a. Stato temporaneo in cui l'attività sta per andare in primo piano.
3. `onResume()` → **In primo piano** e l'utente può interagire.
4. `onPause()` → **Non in primo piano**, ma è **parzialmente visibile** (es: popup, cambio app).
 - a. Quando compare il popup per richiedere un permesso.

5. `onStop()` → **Non visibile**. Può essere distrutta in background.
 - a. Viene premuto il tasto home e l'attività sparisce dallo schermo.
 - b. Quando viene richiesta dall'utente va in started.
6. `onRestart()` → Per riportare un'attività stopped in **primo piano**.
7. `onDestroy()` → sta per essere distrutta permanentemente.

Altre funzioni:

8. `onSaveInstanceState()` → Chiamata prima di una `onDestroy()` serve a salvare lo stato nel bundle object.
9. `onRestoreInstanceState()` → Permette di ricostruire lo stato di una attività precedentemente distrutta partendo dal bundle object.

Il bundle object è un contenitore di **coppie chiave-valore**, può essere passato come argomento a `onCreate()` che viene utilizzato per creare una nuova attività.



Rotazione dello schermo → l'attività viene distrutta e poi ricreata automaticamente con il nuovo layout (*ruotato*).

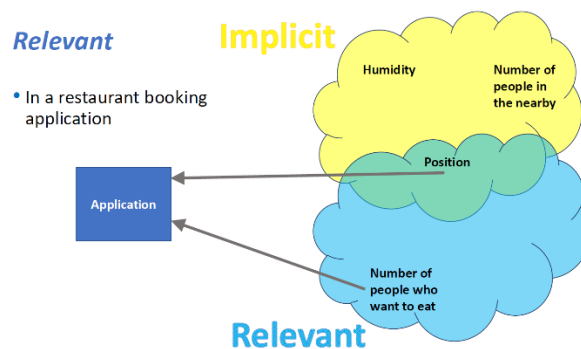
Generalità di Android

Intents

Un messaggio usato da un componente per richiedere un'azione da un'altra applicazione o componente.

Widget GUI

Context-aware computing in generale



Alcuni tipi di contesto sono più importanti di altri: **posizione** (dove), **identità** (chi), **ora** (quando) e **attività** (cosa).

Rappresentazione del contesto

Non esiste un approccio standardizzato. Ma viene costruito basandosi sulle informazioni raccolte dai sensori.

Problematiche del contesto

La context-awareness riguarda la **comprensione dell'intento umano** e spesso sono necessarie molte informazioni sul contesto per comprenderlo:

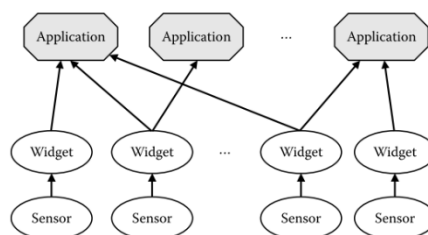
- **Ambiguità:** un utente in piedi di fronte a un dipinto potrebbe non mostrare interesse per l'opera d'arte, potrebbe essere di spalle alla mostra e stare conversando con un amico lì presente.
- **Privacy:** le informazioni di contesto possano essere diffuse in modo inappropriato o a un componente non completamente affidabile,

Spiega i widget e le blackboard. Che cosa si usa in Android?

Widget

Componente software che fornisce alle applicazioni un accesso facile alle rilevazioni dei sensori. Il come un certo fenomeno viene rilevato non deve influire sulla progettazione dell'applicazione.

I widget di contesto isolano le applicazioni da problemi di acquisizione del contesto, **racchiudendo i sensori in un'interfaccia uniforme** in modo da **nascondere all'applicazione la complessità dei sensori** utilizzati.

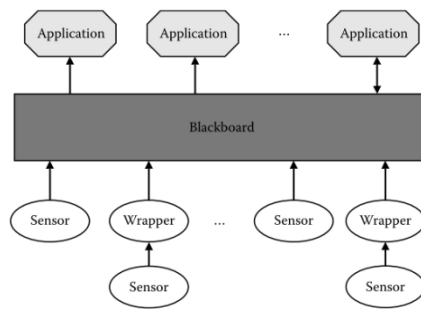


Infatti, vengono sempre fornite informazioni di contesto astratte per soddisfare le esigenze previste delle applicazioni.

Blackboard

I componenti inseriscono informazioni nella blackboard, questa notifica i componenti quando vengono aggiunte informazioni di interesse.

Gli approcci blackboard risultano **inefficienti quando la quantità di dati aumenta** poiché la ricerca di tuple specifiche diventa più difficile.



Android approach

In Android si utilizza un componente software chiamato `SensorManager`, una **classe di sistema** che permette di:

- Accedere ai **sensori fisici** del dispositivo.
- **Listening** → ricevere eventi in tempo reale.

`SensorManager` segue un **approccio simile al blackboard**, perché fornisce un punto centrale di distribuzione del contesto (i dati dei sensori), a cui le app possono registrarsi passivamente e reagire quando il contesto cambia, senza un legame diretto tra produttore e consumatore.

Context awareness e ambiguità, fai un esempio

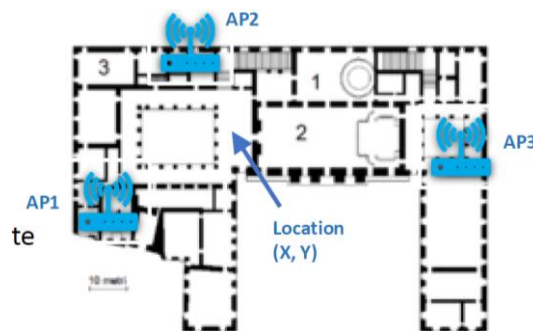
Context awareness → l'applicazione è in grado di **dedurre il contesto** e **adattare il funzionamento**. Ad esempio, lo smartphone capisce che sei in macchina e attiva la modalità "non disturbare".

Ambiguità → Stai viaggiando in autobus o come passeggero in auto, ma il telefono crede che tu stia guidando.

Per gestire l'ambiguità occorre **combinare più fonti diverse dello stesso tipo** (fusione dei sensori) di contesto per migliorare l'accuratezza o l'affidabilità del contesto fornito.

Wifi Fingerprinting

Qual è il principio di funzionamento?



Conosciamo le posizioni di ogni AP e possiamo calcolare la RSSI per ogni AP visibile dal dispositivo.

Viene creata (offline) una tabella dove per ogni posizione (X,Y) si ha il valore dello RSSI osservato da ogni AP. Questa tupla di valori è chiamata **Pre-Recorded Signal Strength** ed è unica per ogni posizione nell'edificio.

Quando un dispositivo vuole **stimare la sua posizione**, calcola tutti gli RSSI rispetto agli AP che vede e poi trova la riga che minimizza la distanza vettoriale tra l'**observed signal strength** (valori che stiamo osservando in questo momento) e il **pre-recorded signal strength** (valori presenti in tabella).

Pre-recorded signal strength					Observed signal strength				
Location (X, Y)	AP1	AP2	AP3	AP4	AP1	AP2	AP3	AP4	
(10, 33)	32	43	54	65	74	11	5	31	
(42, 21)	76	11	4	32					
(5, 12)	-	22	7	43					
(65, 3)	16	19	-	5					
(8, 21)	2	39	25	17					

$$\operatorname{argmin}_i \sum (r_i - s_i)^2$$

Quali sono i vantaggi?

Non richiede particolare hardware, lo RSSI può essere calcolato da qualunque interfaccia di rete e ha un basso consumo. Può essere dispendiosa la creazione della tabella, ma va fatta una volta e non serve aggiornarla spesso.

Quando si usa?

Per localizzare la posizione all'interno di un edificio (il segnale del GPS non arriva all'interno).

Com'è fatta la tabella di questo servizio?

Per ogni posizione all'interno dell'edificio è presente la tupla con i valori degli RSSI rilevati in quella posizione.

Perché i valori sono simili e non esattamente uguali a quelli presenti nella tabella?

Perché il calcolo dello RSSI è soggetto a interferenze, ostacoli mobili (persone, ecc) e dal multipath fading.

Qual è il problema con il multipath?

Il multipath è il fenomeno per cui il segnale **si duplica e arriva al ricevitore lungo percorsi diversi**, a causa di riflessioni, diffrazioni e scattering.

Questo porta all'oscillazione del segnale e quindi alla variabilità nel calcolo dello RSSI. Questo problema è molto presente nell'ambiente indoor e quindi rende questa tecnica meno precisa.

Cosa si usa in Android?

Per la localizzazione si utilizza una API di Google (nuova). Le app che utilizzano la posizione richiedono autorizzazioni dall'utente:

- `ACCESS_COARSE_LOCATION`: precisione nell'ordine di un quartiere.
- `ACCESS_FINE_LOCATION`: precisione nell'ordine più piccolo.
- `ACCESS_BACKGROUND_LOCATION`: quando l'app è in background per consentire maggiore sicurezza per l'utente, esso viene avvisato che alcune applicazioni possono rilevare la posizione in background.

La nuova API fonde le informazioni **provenienti dal GPS e dalla rete** per una migliore precisione e un consumo energetico più efficiente.

Questa nuova API permette di ricevere aggiornamenti (con rateo ridotto) sulla posizione anche quando l'applicazione è in background.

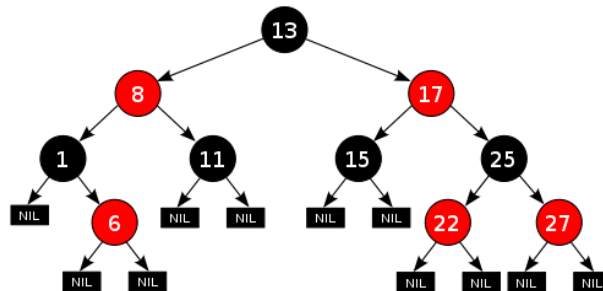
Energia

(NO Q) Scheduler

CFS - Completely Fair Scheduler

Scheduler usato in Linux: decide **equamente** quale processo eseguire sulla CPU e per quanto tempo.

CFS si basa su un albero: ogni cerchio è un processo e il numero indica il **vruntime**: rappresenta quanto tempo ha avuto la CPU (misura le necessità computazionali).



Assegnare la CPU al processo con vruntime minimo (quello che ha avuto meno CPU). Per prendere il vruntime più basso, si sfrutta la proprietà dell'albero binario: prende il nodo più a sinistra nel grafo.

Dopo n unità di tempo di esecuzione, il processo viene reinserito nell'albero con vruntime aumentato di n .

Ogni cambio processo implica un cambio contesto → tempo di esecuzione minimo, entro il quale non può essere tolto dalla CPU.

CFS supporta anche le **priorità**, la quale si trasforma in un peso che viene moltiplicato per il vruntime.

Quantità di carico di un Task

Capacità di un Core

Quante istruzioni al secondo può eseguire un core rispetto al core più potente nel sistema.



Se CPU0 esegue W istruzioni per Hz, mentre CPU1 ne esegue $\frac{W}{2}$. Allora le capacità sono $C_0 = C$, $C_1 = \frac{C}{2}$.

OOP – Operative Performance Point

Coppia di frequenza e tensione a cui una CPU può operare in modo sicuro ed efficiente. Calcolata dal produttore.

Frequency Invariance

L'utilizzazione registrata di un core viene distorta se la frequenza di lavoro varia, questo metodo permette di eliminare questa distorsione. Se un core girasse a **bassa frequenza**, il kernel potrebbe pensare che i task siano "leggeri", anche se in realtà stanno saturando la CPU.

$$Task_{utilization}(p) = duty_{cycle}(p) * \frac{f_{current}}{f_{max}}$$

CPU Invariance

Un sistema può avere molte CPU con capacità diverse. Il carico normalizzato rispetto alla capacità massima del sistema, tenendo conto sia della frequenza attuale che della potenza del core stesso.

$$Task_{utilization}(p) = duty_{cycle}(p) * \frac{c_{current\ cpu}}{c_{max}}$$

CPU and Frequency Invariance

$$Task_{utilization}(p) = duty_{cycle}(p) * \frac{c_{current\ cpu}}{c_{max}} * \frac{f_{current}}{f_{max}}$$

Spiega le energy aware techniques

EAS (livello basso)

Vedi sotto.

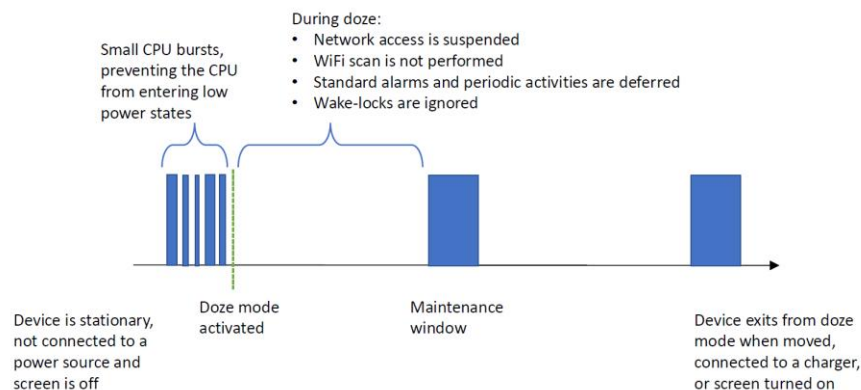
Riduzione delle comunicazioni (livello basso)

Vedi sotto.

Doze mode (livello alto)

Attivata quando il dispositivo rimane non utilizzato per molto tempo e non è collegato all'alimentazione. Il dispositivo esce da questa modalità quando viene alimentato oppure quando viene mosso.

In questa modalità **molti task vengono sospesi e quindi si risparmia molta energia**.



App standby (livello alto)

Le applicazioni utilizzate maggiormente hanno meno restrizioni rispetto a quelle che vengono utilizzate raramente. Le applicazioni vengono classificate in bucket e ad ogni bucket viene assegnata una priorità:

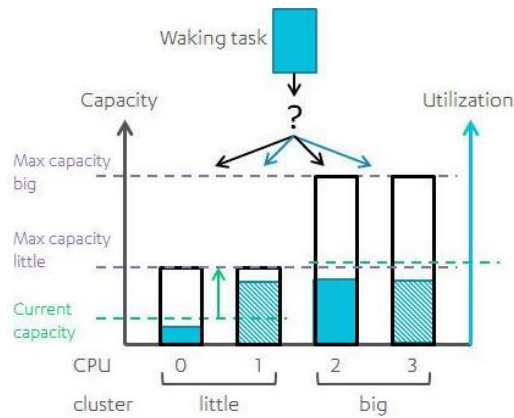
- **Active:** Applicazioni usate in questo momento.
- **Working set:** Applicazioni usate quotidianamente.
- **Frequent:** Applicazioni usate spesso, ma non quotidianamente.
- **Rare:** Applicazioni usate raramente.

EAS in generale

Prende in input il carico generato da ogni task, l'energy model, lo stato attuale delle CPU e restituisce per ogni CPU:

- Assegnazione dei task.
- Frequenza di lavoro.
- Stato di attività.

Vuole sapere dove può girare “abbastanza bene” consumando meno energia possibile.

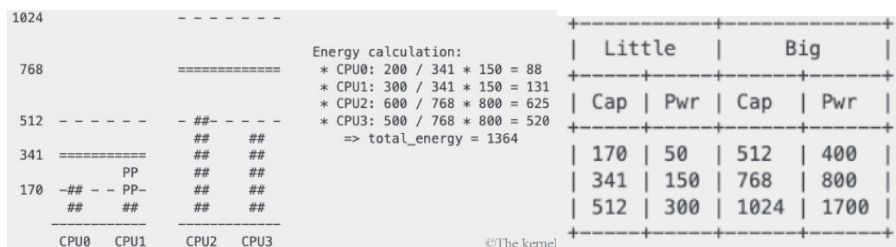


Ci sono quattro CPU di due tipologie diverse (little e big), ognuna con il suo livello di utilizzazione corrente.

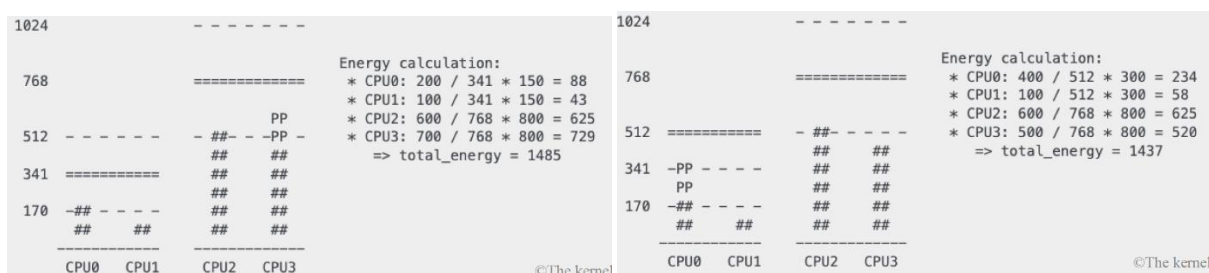
L'energy model indica, per ogni CPU: il consumo energetico per ogni livello di capacità (in genere ci sono tre punti operativi, in ognuno vengono attivati componenti più o meno potenti). In genere lo EM è preso dal file system di Android ed è in un formato simile al JSON.

Dobbiamo stimare, per ogni CPU il consumo energetico se il task venisse spostato dentro di essa per **cercare la soluzione che minimizza l'energia consumata dal sistema** (total energy).

Migrazione in CPU1 → Avanzamento di capacità per CPU1 → `total_energy=1364` → Spostare il task nella CPU1 è la soluzione migliore per risparmiare energia in questo caso.



Migrazione in CPU3 → Avanzamento di capacità per CPU3 → total energy=1485



Rimanere in CPU0 → avanzamento di capacità per CPU0 → total energy=1437

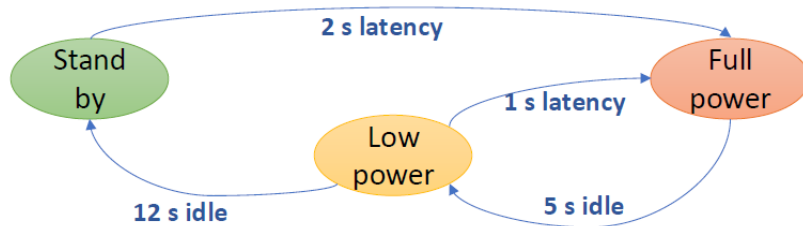
Come si può ridurre il consumo della trasmissione di dati? Quali sono gli stati di un transceiver?

Sfruttando un modello a tre stati:

- **Piena potenza:** I dati trasferibili.
- **Bassa potenza:** I dati non trasferibili, il consumo è del 50%, ma c'è latenza ridotta per tornare alla piena potenza.

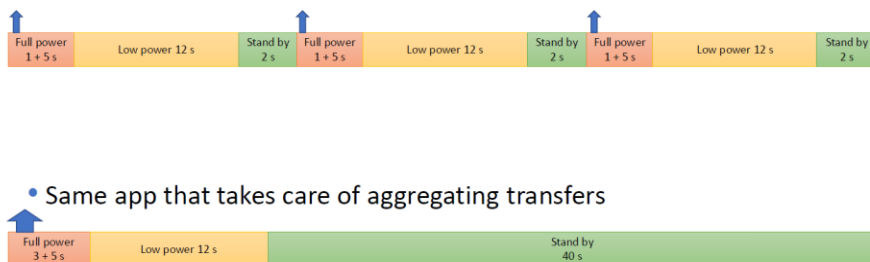
- **Stand by:** I dati non trasferibili, il consumo è minimo, ma ci vuole più tempo per tornare alla piena potenza.

Il cambio di stato è dettato dal tempo di inattività "idle" del modulo di trasmissione.



Data aggregation

Supponiamo che un'applicazione trasferisce dati per 1 s e 3 volte al minuto (non consideriamo la latenza).



Se aggregiamo le informazioni assieme, piuttosto che trasmettere le informazioni ogni 20 s possiamo risparmiare energia perché **li inviamo solo una volta e la radio rimane in full per meno tempo.**

Stessa cosa per il download: invece di scaricare delle entità separatamente, possiamo scaricarle tutte insieme.

Energy status of the phone

Le comunicazioni cellulari e la luminosità sono i fattori di consumo più grandi.

Luminosità adattiva → Usare sensori in grado di misurare la quantità di luce ambientale e sensori di prossimità che rilevano la faccia dell'utente per adattare la luminosità al contesto.

Usare il WiFi invece che cellulare → Il WiFi consuma meno.

Ridurre la potenza erogata ai componenti non utilizzati → per rendere il consumo energetico basso in modalità idle.

Parla di SMP/HMP (symmetric & heterogeneous multiprocessing)

SMP – Symmetric MultiProcessing

In un sistema SMP, tutti i core del processore sono identici. Molto diffuso nei **server** e nei sistemi dove la **coerenza e la prevedibilità** sono essenziali.

- Tutti i core condividono la **stessa memoria** fisica.
- Il carico di lavoro è **distribuito in modo uniforme** tra i core.
- Qualsiasi core può eseguire **qualsiasi tipo di thread** o processo.

Vantaggi:

- **Semplicità di software:** il sistema operativo non deve distinguere tra core diversi.
- **Ottima scalabilità** per carichi omogenei.
- **Facilità di gestione dei thread** e load balancing.

Svantaggi:

- **Consumo energetico elevato:** tutti i core sono sempre "pesanti", anche per task leggeri.

HMP – Heterogeneous MultiProcessing

I core sono **differenti tra loro**, con varie prestazioni e consumi. Il caso più comune è **ARM big.LITTLE**, dove core ad alta potenza sono affiancati da core a basso consumo. Ideale dove è cruciale **equilibrare prestazioni e consumo**, come negli **smartphone**.

Android decide **quale core usare** in base al carico. Vari modi per realizzare questo sistema:

- **Cluster switching**: solo big o solo LITTLE attivi.
- **Core migration**: thread spostati da big a LITTLE e viceversa.
- **Global task scheduling**: tutti i core possono essere attivi contemporaneamente.

L'istruzione set utilizzato da un core grande e da uno piccolo è lo stesso: un task può essere spostato da un core grande ad uno piccolo e viceversa.

Vantaggi:

- **Alta efficienza energetica**: i core LITTLE gestiscono task leggeri, risparmiando energia.

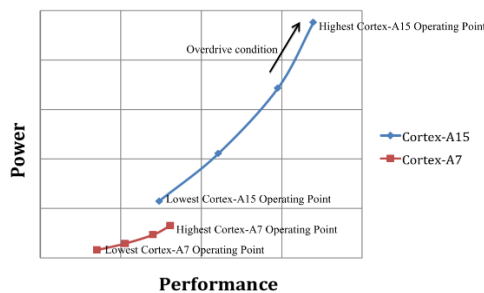
Svantaggi:

- **Maggiore complessità software**: Il SO deve schedulare i task tra i core eterogenei.
- Possibili **latenze** nel passaggio dei thread da un tipo di core all'altro.
- Alcune applicazioni potrebbero **non scalare bene** se non ottimizzate.

Perché HMP aiuta a risparmiare energia?

In un HMP i core non sono uguali: i grandi hanno una potenza maggiore e consumano più energia, mentre i piccoli hanno una potenza minore (frequenza minore), ma consumano meno.

Il punto di HMP è usare i piccoli per i task leggeri lasciando i grandi attivi per meno tempo possibile.



Il grafico mostra la potenza necessaria e il numero di istruzioni al secondo dei core piccoli e grandi.

Si risparmia energia perché la potenza consumata è **proporzionale al quadrato** del voltaggio del processore moltiplicato per la frequenza e la capacità.

$$P = C \times V^2 \times f$$

Spostandoci in un core piccolo $\rightarrow f$ e V vengono ridotti $\rightarrow P$ diminuisce $\rightarrow$ risparmiamo energia.

Come funziona la task migration, illustra le tecniche

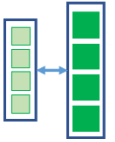
Si tratta di spostare un task in un altro core ed è la base di HMP.

Clustered Switching

n core piccoli e n grandi divisi in **due cluster**. In ogni momento, solo uno dei due è attivo.

Carico aumenta → task migrati ai grandi → piccoli spenti.

Molto **semplice**, ma **inefficiente** → posso usare solo una categoria di core alla volta.



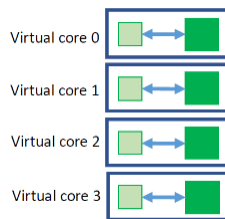
CPU Migration

Core virtuale → composto da un core grande ed uno piccolo. Solo uno dei due può essere attivo.

Carico aumenta → attivo il grande → spengo il piccolo.

Possiamo gestire indipendentemente i core virtuali, possiamo avere il core virtuale 0 con il core grande reale attivo e il core virtuale 1 con il core piccolo reale attivo.

Distribuzione subottimale perché in **ogni core virtuale solo uno dei due core fisici è attivo**.

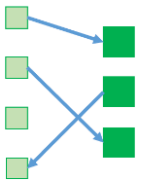


Global Task Scheduling

Non ci sono vincoli sul numero di core piccoli e grandi.

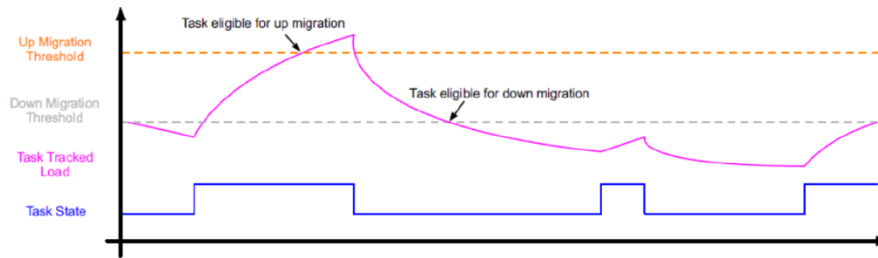
- Task pesante → **spostato** in uno grande libero.
- Task leggero → **spostato** in uno piccolo libero.

Distribuzione ottimale → possiamo avere tutti i core attivi allo stesso tempo, inoltre se ci sono core inattivi, essi possono essere spenti e risparmiare ancora più energia.



Migrazione

La **linea** rappresenta l'attività di un certo task: può essere attivo (1) o in sleep (0).

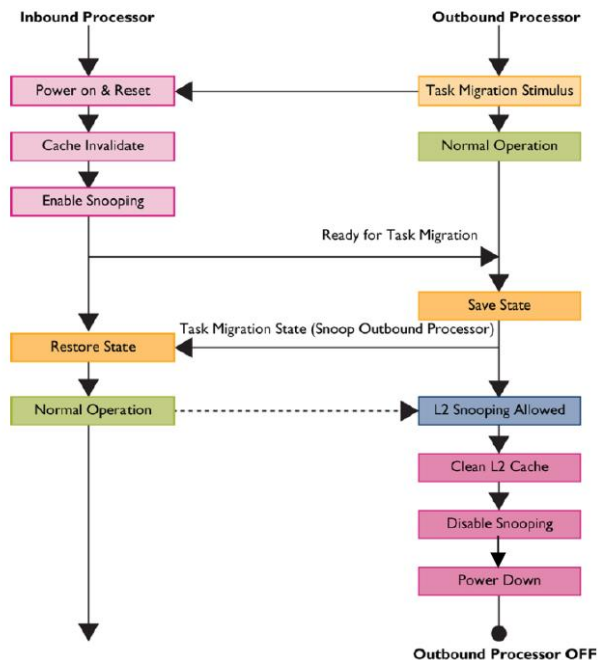


Per ogni task è presente un **contatore**, incrementato quando è attivo e decrementato quando è in sleep.

- Se **contatore** > **Up Migration** → core grande
- Se **contatore** < **Down Migration** → core piccolo

Due soglie per avere più stabilità: non conviene muovere un task frequentemente: richiede un trasferimento del contesto e il flush delle cache.

1. **Stimolo**: L'outbound processor (da cui esce) invia uno stimolo all'inbound processor (in cui viene spostato) per svegliarlo nel caso in cui sia inattivo.
2. **Alimentazione**: Se inattivo, viene alimentato.
3. **Reset dello inbound**: invalida la cache e abilita lo snooping (lettura della cache dell'altro core).
4. **Conferma**: Invia un messaggio all'outbound per indicargli che è pronto.
5. **State Saving**: Lo outbound salva lo stato del task e lo mette nella cache.
6. **Migrazione dello stato**: Copia lo stato del task e lo mette nella propria cache.



Parla della Capacity Awareness

Significa che il task scheduler del SO **riconosce che i core non sono tutti uguali** e tiene conto di:

- **Performance** dei core.
- **Efficienza energetica** dei core.
- **Carico di lavoro richiesto dal processo o thread.**

Senza capacity awareness lo scheduler **tratta tutti i core allo stesso modo**, rischiando di assegnare un task pesante a un LITTLE oppure potrebbe sovraccaricare i big anche per compiti leggeri. **In entrambi i casi sprecherebbe energia.**

Uno scheduler che possiede la capacity awareness è **EAS**.

BLE

BLE in generale

Tecnologia progettata per **ridurre il consumo** e i costi di implementazione. In BLE ogni attore ha uno stato:

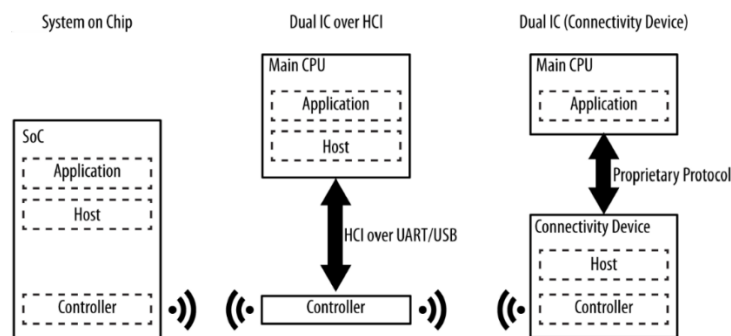
- Ogni dispositivo espone il proprio stato e si comporta come un server (riceve richieste e risponde).
- I client possono **leggere lo stato** (livello di temperatura) oppure **modificare lo stato** di un server (cambiare la temperatura del termostato).
- I server possono **notificare i client** quando lo stato cambia.

Specifiche di BLE

- **Profili:** Regole per i vari tipi di comunicazione tra due dispositivi.
 - **FindMe:** Quando un pulsante viene premuto, causa un allerta su un altro dispositivo.
- **Servizi:** Regole per organizzare i dati nei server.
- **Bluetooth Core Specification:** Definisce il BLE Stack, protocolli e procedure.

BLE Stack

- **Controller.**
 - **Livello Fisico:** Trasmissioni e frequenze.
 - **Livello Link:** Accesso al canale, advertising, scanning, gestione dei pacchetti e cifratura.
- **Host:** dove la maggioranza del protocollo è implementato.
 - L2CAP, ATT, GATT, SMP, GAP.
- **Applicazione:** Logica utente, sfrutta lo stack per interagire con il sistema BLE.



- **Hardware:** I tre componenti sono implementati interamente in hardware in un system-on-chip.
- **Software:** I tre componenti sono interamente in software, poco comune.

BLE connection events

Due dispositivi comunicano solo durante gli eventi di connessione → dopo ritornano in sleep, per risparmiare energia → comunicazione è composta da burst di dati.

Connection interval → tempo (7, 5ms – 4s) in cui i dispositivi stanno in sleep tra due eventi di connessione. Più è grande e più energia si risparmia ma minore è il throughput.

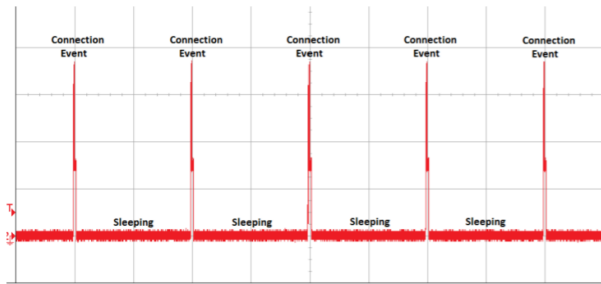


Figure 1- Current Consumption versus Time during a BLE Connection

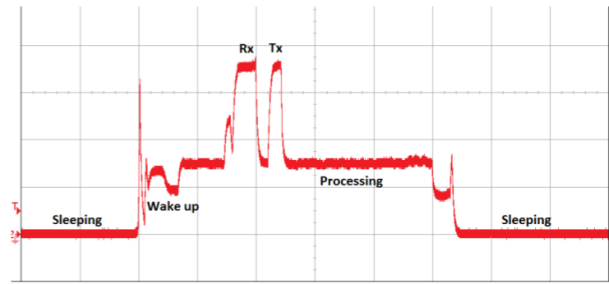


Figure 2- Current Consumption versus Time during a single Connection Event

BLE Roles

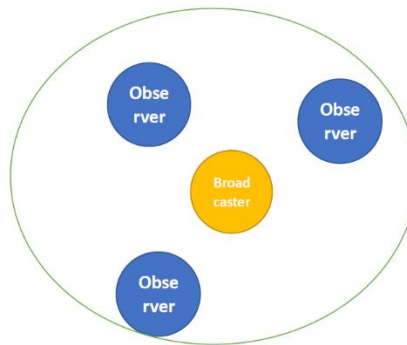
I dispositivi tipicamente assumo una delle due coppie di ruoli:

Broadcaster/Observer

Unidirezionale: 1 broadcaster → n observers.

L'unica modalità per trasmettere qualcosa a **più ricevitori contemporaneamente**.

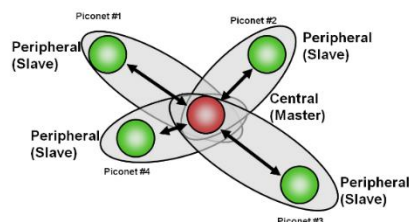
- Il broadcaster periodicamente invia **non-connectable advertising** (gli observers non possono stabilire una connessione col broadcaster).
- Gli observer scansionano tutte le frequenze per ricevere un advertising.



Master/Slave

Bidirezionale: 1 master → 1 slave. Viene creata una connessione, quindi è ottima per ripetuti scambi bidirezionali.

1. Lo slave invia **connectable advertising** in broadcast.
2. Il master fa **scanning**, ovvero scansiona tutte le frequenze.
3. Il master riceve l'advertising e viene stabilita una connessione con lo slave.
4. Lo slave dopo aver ricevuto la conferma non invia più advertising.



Un dispositivo può essere sia master che slave allo stesso tempo ma rispetto a nodi diversi.

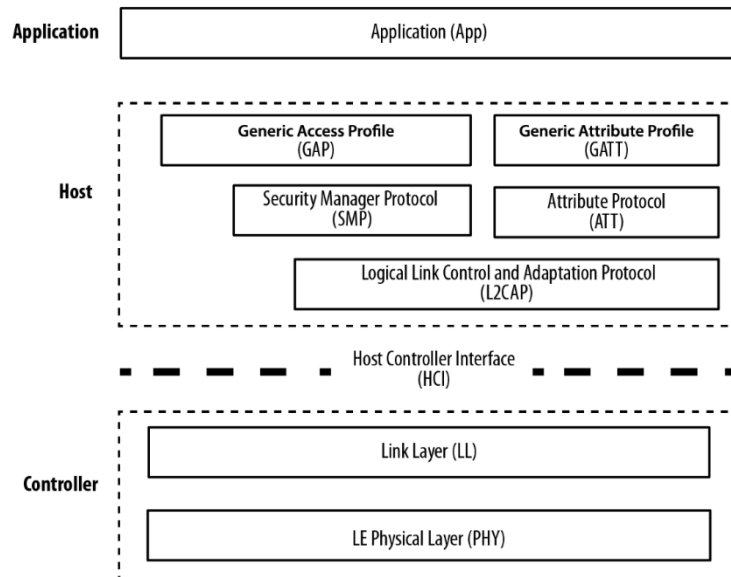
Nonostante sia **più complesso** è **più efficiente**, perché due coppie di nodi possono stabilire il connection interval da utilizzare in modo da risparmiare energia.

Come mai BLE consuma poco?

- **Troughput basso** teoricamente $< 2 \text{ Mbps}$. Utilizzato (10–100 Kbps).
- **Range piccolo** teoricamente $< 30 \text{ m}$. Utilizzato (5m).
- **Ampio connection interval** (7, 5ms – 4s) $\rightarrow$ i dispositivi stanno in idle per molto tempo.

Dati inviati in burst durante gli eventi di connessione. Tra due eventi (connection interval) il dispositivo è in sleep.

Parla di tutti i layer e del meccanismo di connessione



Controller

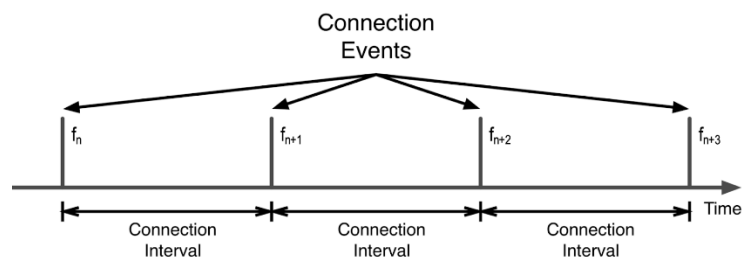
Livello Fisico

Gestisce le trasmissioni e il frequency hopping. Utilizza la banda ISM 2,4 GHz (WiFi). Suddivisa in **40 canali**: 37 per i dati, gli ultimi 3 per advertising.

Frequency hopping $\rightarrow$ Per calcolare il prossimo canale da utilizzare dopo aver trasmesso viene utilizzata la formula:

$$f_{n+1} = |f_n + hop|_{37}$$

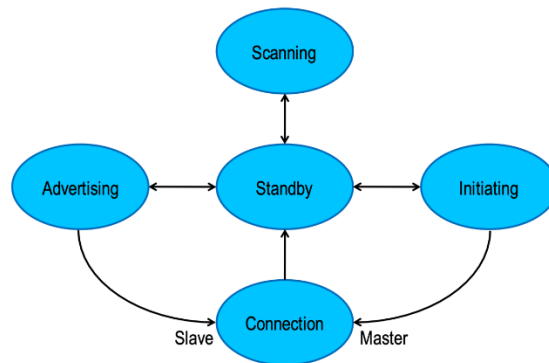
hop è deciso al momento della connessione tra due dispositivi.



Livello Link

Gestisce molte cose:

- **Ruoli:**
 - **Advertiser:** invia advertising.
 - **Scanner:** riceve advertising.
 - **Master (Central):** Invia le richieste di connessione e le gestisce.
 - **Slave (Peripheral):** Accetta richieste di connessione seguendo il timing del master.
- **Formato dei frame.**
- **Formato degli indirizzi.**
 - **Public device address:** assegnato dal costruttore e registrato con IEEE, non cambia mai ed è unico al mondo.
 - **Random device address:** può essere programmato o generato in runtime.

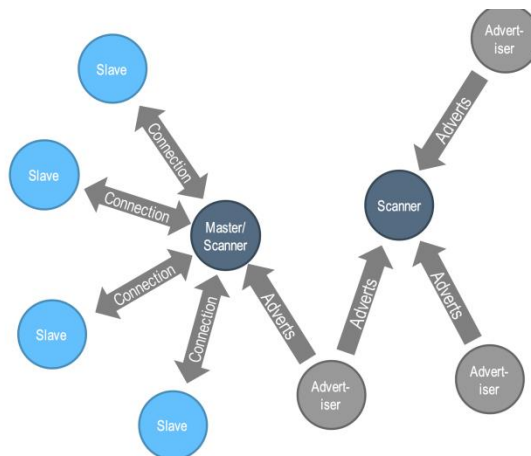


Gli advertisement sono inviati ad un intervallo fisso che varia tra i 20ms ai 10s.

Richiesta di connessione

Inizialmente avremo un master connesso a vari slave.

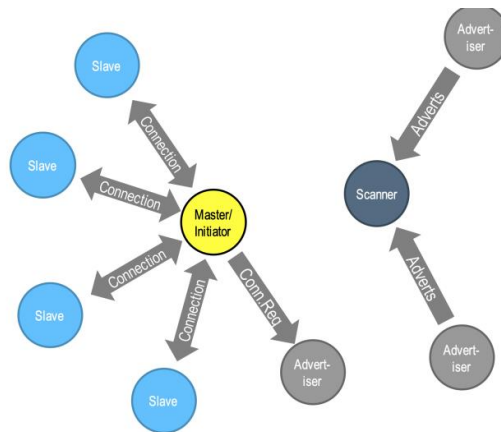
1. Si aggiunge un nuovo nodo, inizialmente è nello stato Advertiser.



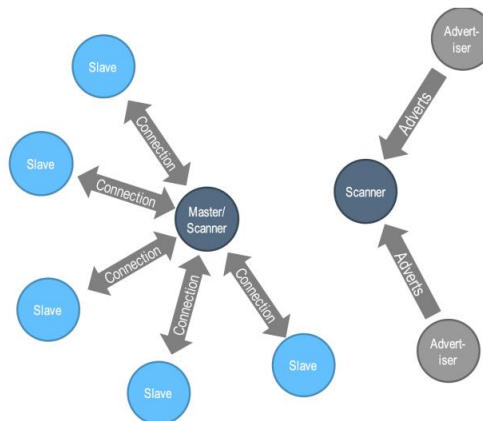
2. Il nodo in quanto Advertiser, invia degli advertising, ricevuti dai due scanner in prossimità.

Uno dei due scanner è anche master, quindi può inviare richieste di connessione.

3. Il master, quindi, va in stato Initiator e invia la richiesta di connessione.



4. Lo Advertiser riceve la richiesta di connessione e la accetta diventando lo slave del master.



Host

L2CAP Logical Link Control and Adaptation Protocol (L2CAP).

- Multiplexing/demultiplexing da/verso i livelli superiori.
- Frammentazione/riassemblaggio.
- L'intestazione del pacchetto L2CAP occupa quattro byte, il che significa che la lunghezza effettiva del payload utente è di $27 - 4 = 23$ byte.
- Interagisce con i protocolli di livello superiore.

Attribute Protocol (ATT)

Protocollo client/server stateless basato sugli attributi presentati da un dispositivo.

I dati sono organizzati sotto forma di attributi.

Ogni attributo ha alcune proprietà, ha un handle di 16 bit, un UUID di 128 bit per identificare le informazioni contenute, comprende un insieme di permessi e il valore dell'attributo stesso.

Ogni attributo ha un handle di 16 bit, ogni volta che il client vuole leggere/scrivere un attributo, deve specificare l'handle number.

Handle	Value
0x0009	0x54656d70657261747572652053656e736f72
0x0022	0x04
0x0098	0x0802

Gli attributi hanno un tipo, il tipo è l'UUID che determina il significato del valore.

GAP

Definisce i ruoli:

- Broadcaster/observer, central/peripheral
- Discoverable/non discoverable
- Connectable/non connectable

Definisce i metodi d'invio dati, scoprire i dispositivi e gestire connessioni.

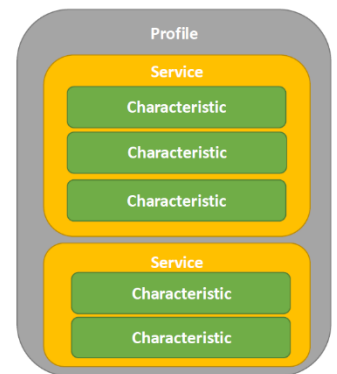
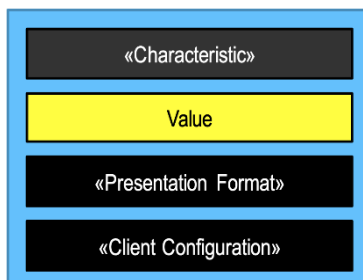
GATT

Definisce un modello di dati e procedure per scoprire, leggere, scrivere e inviare i dati.

- I dati sono organizzati in servizi.
- I servizi sono organizzati in profili.

Servizio: fornisce una o più caratteristiche identificate da un UID.

Una caratteristica ha una riga che indica la dichiarazione, un valore e righe aggiuntive per il descrittore. Incapsula i dati, è un valore con un tipo noto e un formato noto definito nelle characteristic specification.



Sensori

Quali tipi di sensori ci sono?

I sensori Android possono essere classificati come:

- **Ambientali:** forniscono temperatura, pressione atmosferica, ecc.
- **Movimento.**
- **Orientamento.**

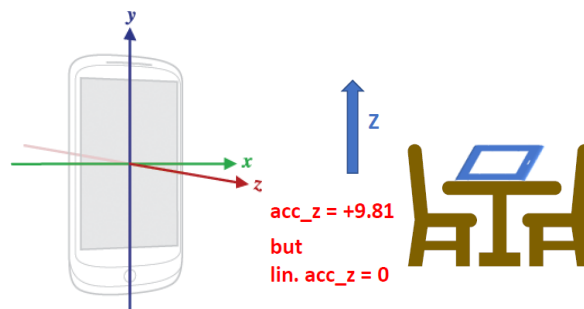
Problema → i sensori variano tra i dispositivi e la loro disponibilità dipende dal livello API. Alcuni sono sempre disponibili, mentre altri no.

In Android per accedere ai sensori si usa `SensorManager`.

Movimento

- **Accelerometro:** restituisce l'accelerazione.
- **Giroscopio:** restituisce la velocità angolare.
- **Vettore di rotazione.**
- **Gravità:** sensore software che restituisce la direzione dell'accelerazione di gravità.
- **Rilevamento dei passi:** viene generato un evento per ogni passo.

Scegli un sensore qualsiasi e dimmi cosa fa



Accelerazione Lineare

Viene sottratta la gravità

- **Fermo a faccia in su:** $acc_x = acc_y = 0$, $acc_z = 0$.
- **Caduta libera** (faccia in giù): $acc_x = acc_y = 0$, $acc_z = -g$.
- Accelerazione a verso l'alto a faccia in su: $acc_x = acc_y = 0$, $acc_z = a$

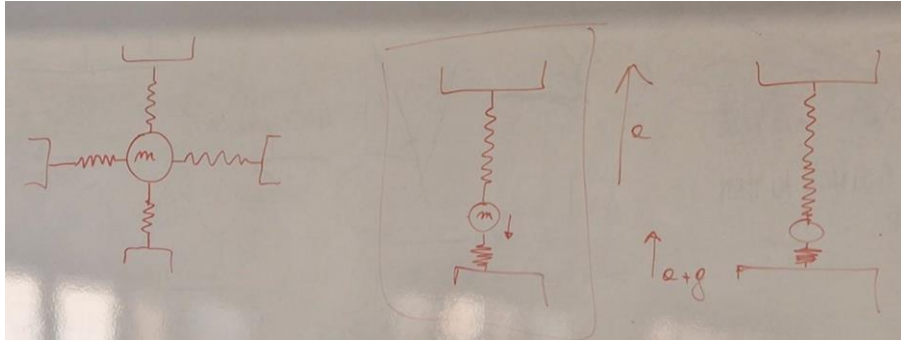
Accelerazione Totale

Include la gravità come offset.

`accelerazione = acc. Lineare + gravità`

L'accelerometro può essere visto come una massa connessa a 4 molle

Se il device è appoggiato ad un tavolo e se ci concentriamo sull'asse verticale, la molla che sta sotto la massa è compressa e la molla sopra è allungata (dato che si è trovato un punto di equilibrio, la molla allungata sta esercitando una forza pari a g).

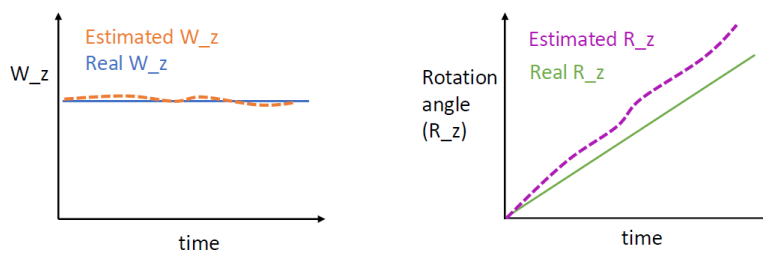


Se spostiamo la massa verso l'alto con un'accelerazione a , l'accelerazione totale a cui è sottoposta è $g+a$ (g data dalla molla allungata e a data da noi).

- **Fermo a faccia in su:** $\text{acc}_x = \text{acc}_y = 0$, $\text{acc}_z = g$.
- **Caduta libera** (faccia in giù): $\text{acc}_x = \text{acc}_y = 0$, $\text{acc}_z = 0$ (circa).
- Accelerazione a verso l'alto a faccia in su: $\text{acc}_x = \text{acc}_y = 0$, $\text{acc}_z = g + a$

Giroscopio

Restituisce la velocità angolare e non l'accelerazione. Per calcolare l'angolo di rotazione, la velocità angolare deve essere integrata (nel senso di calcolare l'integrale).



Vettore di rotazione

Restituisce l'orientamento rispetto al sistema di riferimento globale della Terra. Ottenuto dalla **fusione di giroscopio, accelerometro, campo geomagnetico**.

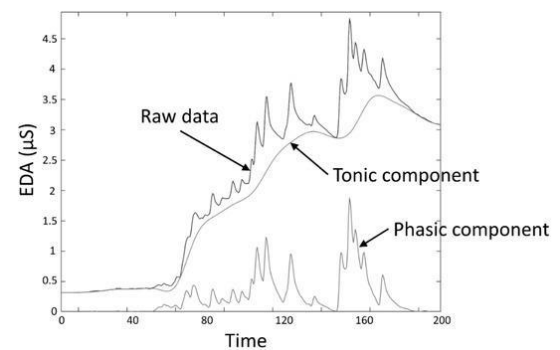
- **Azimut:** angolo attorno all'asse z , è diverso da 0, se non punta verso Nord.
 - Giroscopio + Campo geomagnetico.
- **Beccheggio:** angolo attorno all'asse x (abbassare o alzare il muso di un aereo).
 - Giroscopio + Accelerometro.
- **Rollio:** angolo attorno all'asse y .
 - Giroscopio + Accelerometro.

Il giroscopio può anche essere utilizzato per rendere la stima più accurata perchè, se utilizziamo la gravità dobbiamo applicare un filtro passa basso all'informazione, con il giroscopio possiamo integrare le informazioni di beccheggio e rollio per ottenere valori più accurati.

EDA - ElectroDermal Activity

La conduttanza elettrica registrata dalla pelle aumentata che si verifica durante l'eccitazione fisiologica/emotiva a causa dell'attivazione delle ghiandole sudoripare.

- **Sistema Nervoso Autonomo (SNA)** regola i processi fisiologici involontari.
- **Sistema Nervoso Simpatico:** parte del SNA responsabile della risposta "combatti o fuggi" (situazione di pericolo che comporta aumento della frequenza cardiaca, stimolazione del sudore, dilatazione delle pupille).



Il livello di conduttanza cutanea (EDA) viene misurata in microSiemens [S], l'inverso dell'Ohm.

La EDA è un segnale composto da: **componente tonico** e **componente phasic**. In genere eventi stressanti provocano dei picchi (phasic component) nel componente tonico. Noi vogliamo valutare questi picchi.

Il componente tonico è ottenuta con un LPF → $Phasic = Raw - Tonic$

Se vogliamo utilizzare uno smartphone come step counter, spiega una possibile procedura utilizzando l'accelerometro.

Tramite dati rilevati dall'accelerometro: camminare produce un movimento lungo i tre assi del corpo.

Simple Step Detection Algorithm

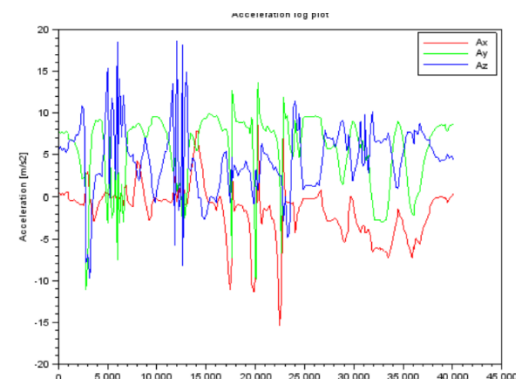
Fase 1: Smoothing

Non sappiamo come lo smartphone è orientato → bisogna considerare tutti gli assi.

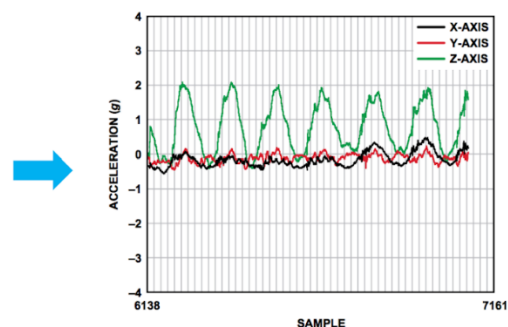
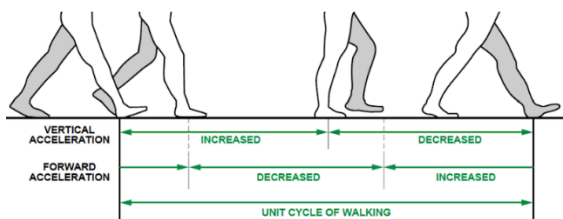
Il segnale dall'accelerometro è rumoroso:

1. Dispositivo non aderente al corpo → movimenti all'interno della tasca.
2. Ogni movimento è leggermente differente dal movimento precedente.

Per mitigare il rumore → **smoothing con media a finestra mobile** di dimensione maggiore o uguale a 3 (sostituisco ogni campione con la media dei due precedenti e sé stesso).



Fase 2: Rilevamento della soglia dinamica



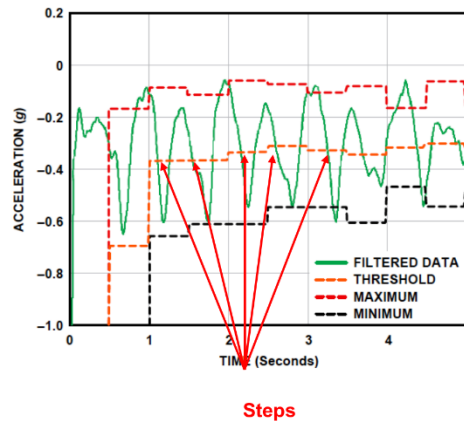
La camminata provoca un picco periodico su uno degli assi, per rilevarlo usiamo una soglia dinamica. *Perché dinamica?*

- **Variabilità del movimento del soggetto:** il segnale varia in base anche all'umore.
- **Variabilità inter-subject:** ogni soggetto è differente.

La soglia viene applicata sull'asse con il picco più alto (si considera picco se si supera la soglia).

Soglia dinamica: $\frac{\max + \min}{2}$ → *min* e *max* aggiornati ogni 50 campioni.

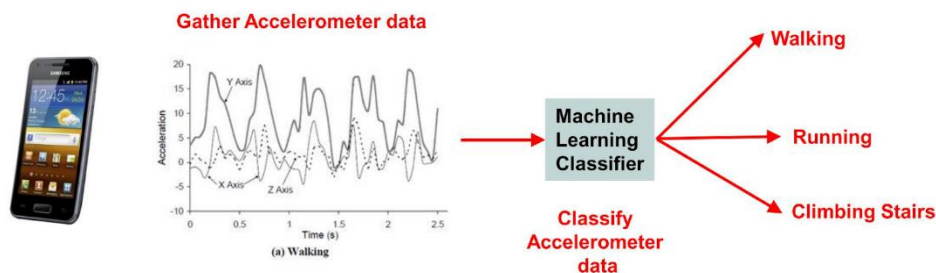
Gradino → **segnale** attraversa **soglia durante una pendenza negativa** (*campione_i* < *campione_{i-1}*).



ML Step Detection

L'algoritmo di rilevamento del passo appena descritto è molto semplice, nella realtà vengono usati algoritmi più sofisticati che sfruttano anche l'intelligenza artificiale.

1. Prendiamo il segnale (3 segnali, uno per ogni asse), il quale essendo stato campionato e etichettato (supervised training) comporrà il dataset per il classificatore.
2. Lo dividiamo in finestre temporali di [1-2]s.
3. Calcoliamo l'accelerazione media negli assi e le altre feature durante queste finestre.
4. Le feature misurate vengono utilizzate per allenare un sistema in modo che riconosca le varie attività.



L'orientazione del dispositivo è un problema? Se sì, come si risolve?

Può essere un problema nel rilevare i passi. Per mitigare il problema si effettua uno **smoothing del segnale** per rimuovere vibrazioni e/o piccoli movimenti fatti nella tasca.

Poi, dato che non si può sapere l'orientamento del telefono, vengono considerati tutti e tre gli assi, in particolare quello che mostra i picchi più grandi.

Quali sono gli altri problemi quando contiamo il numero di passi?

Le **vibrazioni** prodotte dai rumori esterni (motori o altro) potrebbero essere conteggiate come **passi**.

Il movimento della camminata è periodico, possiamo sfruttare questa cosa per eliminare meglio il rumore esterno.

- **Correre** → 4 passi/s → 0,25s per passo.

- **Camminare lentamente:** 1 passo/s → 1s per passo.

Per mitigare l'effetto delle vibrazioni basta eliminare gli attraversamenti negativi della soglia dinamica che si verificano al di fuori del periodo [0,25s , 1s].

Se l'utente ha appena fatto un passo, e so che l'utente fa 1 passo/s, allora posso considerare rumore un attraversamento fatto 0,1s dopo il passo.